Forge: Gym Tracker

Software Engineering Documentation

Software Requirements Specification, Component Specification, component breakdown and UML diagrams

Version 1.0 | Component-based architecture

Contents

Part 1. Software Requirements Specification 3
1. Introduction 3
2. Overall Description 3
3. System Architecture Overview 4
4. Functional Requirements 4
5. External Interface Requirements 6
6. Non-Functional Requirements 7
7. Data Requirements 7
8. Business Rules 8
9. Future Requirements 8
Appendix A. Use Case List 8
Appendix B. Requirement to Component Map 9

Part 2. Component Specification 10
1. Overview 10
1.1 Provides and requires at a glance 10
1.2 Interface catalogue 10
1.3 Collection ownership 10
2. Component Details 11
2.1 C1 Identity & Profile 11
2.2 C2 Data Store 11
2.3 C3 Exercise Library 12
2.4 C4 Workout Logging 13
2.5 C5 Progress & Analytics 14
2.6 C6 Weekly Plan 14
2.7 C7 Attendance & Goals 15
2.8 C8 Body Tracking 16
2.9 C9 InBody Reports 16
2.10 C10 Units & Dates (Shared Kernel) 17
2.11 C11 UI Kit 18
2.12 C12 App Shell & Composition Views 18
2.13 C13 Crew & Leaderboard 19

Part 3. Component Breakdown and Ownership 20

Part 4. UML Diagrams 22
Use Case diagrams 23
Component diagrams 28
Class diagrams 32
Sequence diagrams 39

Part 1. Software Requirements Specification

What Forge shall do and the qualities it shall have.

1. Introduction

1.1 Purpose

This document specifies what the Forge application shall do and the qualities it shall have. It is written for the product owner, the developers and the reviewers, and it is the reference for the UML models and for acceptance testing.

1.2 Scope

Forge is a web application in which each athlete keeps a private space with their own data: exercise library, weekly workout plan, logged sets, gym attendance, body weight and InBody body-composition reports. The product is built from independent components (Section 3) that talk to each other only through defined interfaces.

In scope: everything in Sections 4 to 7. Out of scope for version 1.0: the crew leaderboard (Section 9), challenges and badges, progress photos and Arabic/RTL support. The separate TRIO FIT application is not covered.

1.3 Definitions and abbreviations

Term	Meaning
Athlete	A registered user of Forge.
Set	One logged effort of an exercise: weight, reps, date.
PR	Personal record: a set whose weight beats every earlier set of that exercise.
Plan	The weekly workout plan with seven days, shown Saturday to Friday.
Gym day	A day with a logged set or a manual check-in.
Weekly goal	Number of training days a week the athlete aims for (taken from the plan).
Component	A self-contained part of the system with provided and required interfaces.
Composition View	A page that combines several components and contains no business rules.
InBody	A body-composition scan report (muscle, fat, BMR).
SRS / UML	Software Requirements Specification / Unified Modeling Language.
FR / NFR	Functional / non-functional requirement.

1.4 References

- Project README and conventions (CLAUDE.md).
- Free Exercise DB, public-domain exercise dataset (github.com/yuhonas/free-exercise-db).
- Firebase Authentication and Cloud Firestore documentation.

1.5 Document overview

Section 2 describes the product in general. Section 3 defines the components. Section 4 lists functional requirements per component. Sections 5 to 8 cover interfaces, quality attributes, data and business rules. Section 9 lists planned work. Appendix A lists the use cases and Appendix B maps requirements to components.

2. Overall Description

2.1 Product perspective

Forge is a single-page web application that runs in the browser. It has no server code of its own: authentication and storage are provided by Firebase, and the app is delivered by Vercel. It also reads exercise images from a public CDN and opens YouTube search pages in a new tab.

2.2 Product functions (summary)

- Create an account and sign in with email or Google.
- Browse and search a library of 584 exercises organised by muscle, with images and steps.
- Build a weekly plan, edit sets and reps, and reshape the week with shift, move, swap, copy, clear and undo.
- Log sets quickly with prefilled values, a sets-done counter, PR detection and a rest timer.
- Follow progress per exercise and on charts.
- Track gym attendance, the weekly goal and the week streak.
- Track body weight against a goal and record InBody reports.

2.3 User classes

User class	Description
Athlete	The only human actor. Trains regularly, uses a phone in the gym and a desktop sometimes. Has no technical knowledge.
Crew member (future)	An athlete who opts in to share progress with friends (Section 9).

External systems: Firebase Authentication (including Google sign-in), Cloud Firestore, the exercise image CDN (jsDelivr with a GitHub raw fallback) and YouTube (link only).

2.4 Operating environment

Current evergreen browsers (Chrome, Safari, Firefox, Edge) on iOS, Android, Windows and macOS. An internet connection is required to sign in and sync.

2.5 Design and implementation constraints

- The front end is React 18 with Vite, plain JavaScript and plain CSS.
- Persistence is Firestore only; images are stored as compressed data inside documents (no Firebase Storage).
- Everything must remain free: no paid service.
- All weights are stored in kg, dates as YYYY-MM-DD, and weeks start on Saturday.
- Security rules are published manually in the Firebase console.
- The code is organised into the components of Section 3.

2.6 Assumptions and dependencies

- Users have a modern browser and an internet connection.
- The Free Exercise DB content stays available under its public-domain license; images have a fallback source.
- Firebase free tier limits are enough for a small group of friends.

3. System Architecture Overview

Forge is component-based, not monolithic. Rules: (R1) one responsibility per component; (R2) every Firestore collection has exactly one owning component and others use its interface; (R3) dependencies have no cycles; (R4) pages that combine components contain no business rules. Dependency direction is: App Shell, then feature components, then Data Store, Units & Dates and UI Kit.

ID	Component	Responsibility	Provided interfaces
C1	Identity & Profile	Sign in/out, session, profile (name, unit, weight goal, weekly goal).	IAuth, IProfile
C2	Data Store	The only code that talks to Firestore; live subscriptions.	IDataStore
C3	Exercise Library	Dataset, search, muscle folders, media, favorites, custom exercises, user folders.	IExerciseCatalog, IExerciseSearch
C4	Workout Logging	Logging sets, sets-done counter, rest timer, drafts, history.	ILogger, ILogReader
C5	Progress & Analytics	PRs, trends, best weight, chart data.	IProgress
C6	Weekly Plan	7-day plan, templates, week editing, undo, legacy schedule migration.	IPlan
C7	Attendance & Goals	Gym days, check-ins, weekly goal, week streak.	IAttendance
C8	Body Tracking	Weigh-ins and weight-goal progress.	IBodyTracker, IBodyReader
C9	InBody Reports	Body-composition reports, comparison, image compression.	IInBody, IInBodyReader
C10	Units & Dates (Shared Kernel)	kg/lb conversion, rounding, date strings, Saturday-first week math.	IUnits, IDates
C11	UI Kit	Reusable interface parts and theme (Sheet, Modal, Stepper, Undo toast, PlateBar, chart card).	UI components
C12	App Shell & Composition Views	Routing, navigation and pages that combine components without business logic.	Top level
C13	Crew & Leaderboard (planned)	Opt-in sharing and ranking among friends.	ICrew

Component ownership of every source file is recorded in the design note docs/design/01-components.md. The Component Diagram (separate file) shows the provided and required interfaces.

4. Functional Requirements

Priority: M = must have, S = should have. "UC" refers to the use cases in Appendix A. The word "shall" marks a requirement.

4.1 C1 Identity & Profile

Sign in/out, session, profile (name, unit, weight goal, weekly goal).

ID	Requirement	Pri.	Use case
FR-ID-01	The system shall let a visitor create an account with a display name, email and password.	M	UC-01
FR-ID-02	The system shall let a user sign in with email and password.	M	UC-02
FR-ID-03	The system shall let a user sign in with a Google account.	M	UC-02
FR-ID-04	The system shall let a signed-in user sign out, after which no private data is shown.	M	UC-03
FR-ID-05	On the first sign-in the system shall create the profile with a name (from the account or the email prefix), unit kg, and empty weight goal.	M	UC-01, UC-02
FR-ID-06	The system shall let the user edit the display name, the unit (kg or lb) and the default gym days per week.	M	UC-04
FR-ID-07	Routes other than sign-in shall be available only to signed-in users; an anonymous visitor shall see the sign-in screen instead.	M	UC-02

4.2 C2 Data Store

The only code that talks to Firestore; live subscriptions.

ID	Requirement	Pri.	Use case
FR-DS-01	All user data shall be stored under a single per-user root (users/{uid}) and never be mixed between users.	M	All
FR-DS-02	The system shall provide live collection subscriptions so every screen updates automatically when data changes.	M	All
FR-DS-03	The system shall support create, replace, merge, update and delete of a document by id, and create-if-missing that never overwrites an existing document.	M	All
FR-DS-04	Every created document shall carry a server timestamp (createdAt).	S	All

4.3 C3 Exercise Library

Dataset, search, muscle folders, media, favorites, custom exercises, user folders.

ID	Requirement	Pri.	Use case
FR-LIB-01	The system shall include a bundled library of 584 strength exercises (public-domain Free Exercise DB), loaded on demand so it does not slow the first page load.	M	UC-05
FR-LIB-02	The system shall organise the library into 17 muscle folders, each with a name and emoji, plus a "My custom" folder.	M	UC-05
FR-LIB-03	The system shall search the library by name with synonym support and rank popular exercises first.	M	UC-05
FR-LIB-04	The system shall filter exercises by equipment group and by "popular".	S	UC-05
FR-LIB-05	The system shall show, for each library exercise, primary and secondary muscles, equipment, mechanic, start and end images and numbered steps.	M	UC-06
FR-LIB-06	Images shall load from a CDN with an automatic fallback source; a placeholder shall be shown if both fail.	S	UC-06
FR-LIB-07	The system shall provide a "Watch how-to" button that opens a YouTube search for the exercise. It shall not embed or store videos.	M	UC-06
FR-LIB-08	The system shall let the user mark and unmark favorites and shall list favorites.	S	UC-07
FR-LIB-09	A library exercise shall get a personal document (id lib_<datasetId>) only when it is first favorited, logged or added to the plan; creating it twice shall have no effect.	M	UC-07, UC-13, UC-10
FR-LIB-10	The system shall let the user create a custom exercise (name, muscle folder, equipment), rename it, archive or restore it, and delete it together with its logs after confirmation.	M	UC-08
FR-LIB-11	The system shall let the user create, rename and delete personal folders, add existing or new exercises to a folder, and keep an exercise in several folders. Deleting a folder shall never delete exercises or logs.	S	UC-09
FR-LIB-12	The system shall show recently logged exercises for quick access.	S	UC-05

4.4 C4 Workout Logging

Logging sets, sets-done counter, rest timer, drafts, history.

ID	Requirement	Pri.	Use case
FR-LOG-01	The system shall let the user log a set with weight, reps and date; weight shall be stored in kg.	M	UC-13
FR-LOG-02	The quick logger shall prefill weight and reps from the last session and offer plus/minus steppers.	M	UC-13
FR-LOG-03	The quick logger shall write one log document per set with the fields exerciseId, exerciseName, weight, reps, sets and date.	M	UC-13
FR-LOG-04	The quick logger shall show a sets-done counter for today against the planned target (for example 2/3).	M	UC-13
FR-LOG-05	The system shall announce a new personal record when a set beats every earlier set of that exercise (not for the first ever set).	M	UC-13
FR-LOG-06	The system shall offer an optional 90-second rest timer after each set and remember the user's choice.	S	UC-13
FR-LOG-07	Values typed but not logged shall be kept per exercise when the logger is closed and reopened in the same session.	S	UC-13
FR-LOG-08	The exercise page shall allow a detailed entry (weight, reps, number of sets, date not in the future) and list the history with delete.	M	UC-13
FR-LOG-09	Logging an exercise that is not in today's plan shall be possible from the Home "Do something else today" action.	S	UC-13

4.5 C5 Progress & Analytics

PRs, trends, best weight, chart data.

ID	Requirement	Pri.	Use case
FR-PRG-01	The system shall show for each exercise the last weight, the best weight and a trend (up, down or steady) comparing the latest session with the average of the previous three; a change under 2% is steady.	M	UC-14
FR-PRG-02	The system shall mark personal records in the history and count them per exercise.	M	UC-14
FR-PRG-03	The system shall show a compact chart of the top weight over the last eight sessions and the last five sets.	M	UC-14
FR-PRG-04	The system shall show a one-line summary (last, best, trend) wherever an exercise is listed in the plan or on Home.	S	UC-14, UC-15
FR-PRG-05	The Charts page shall plot body weight, each InBody metric, gym days per week, training volume per week, and for each exercise top weight and estimated 1-rep max.	S	UC-20
FR-PRG-06	New screens shall not display the estimated 1-rep max (it remains only on the Charts page).	S	UC-14

4.6 C6 Weekly Plan

7-day plan, templates, week editing, undo, legacy schedule migration.

ID	Requirement	Pri.	Use case
FR-PLN-01	The system shall hold one plan with seven day entries, displayed Saturday to Friday, each with an optional label, a rest flag, focus muscles and an ordered list of exercises.	M	UC-10
FR-PLN-02	The user shall be able to add an exercise to a day (with sets and reps), remove it, and reorder it. The same exercise shall not be added twice to a day.	M	UC-10
FR-PLN-03	Sets and reps targets shall be editable at any time directly on the exercise row.	M	UC-10
FR-PLN-04	When adding an exercise the system shall suggest popular exercises for the day's focus muscles, recently used exercises, and offer search of the whole library.	S	UC-10
FR-PLN-05	Marking a day as rest shall keep its exercises so they return if the flag is cleared.	M	UC-10
FR-PLN-06	The system shall provide ready-made starter plans (for example Push/Pull/Legs) that replace the week after confirmation.	S	UC-12
FR-PLN-07	The system shall shift the whole week by N days forward or backward with wrap-around and show a before/after preview.	M	UC-11
FR-PLN-08	The system shall move, swap or copy a whole day to another day, and clear all exercises of a day at once (keeping label, focus and rest flag).	M	UC-11
FR-PLN-09	Every week-editing action and the removal of an exercise shall offer an Undo for a few seconds that restores the previous plan exactly.	M	UC-11
FR-PLN-10	Week-editing actions shall write only the days that changed and shall report an error without partial loss if saving fails.	M	UC-11
FR-PLN-11	The system shall migrate an existing old schedule into the plan once, leaving the old documents untouched, and tell the user about custom days that had no weekday.	M	UC-10
FR-PLN-12	The system shall let the user add an exercise to today's plan in one tap (turning off the rest flag if needed).	S	UC-21
FR-PLN-13	Tapping a plan exercise shall open the exercise sheet on the Progress tab.	M	UC-14

4.7 C7 Attendance & Goals

Gym days, check-ins, weekly goal, week streak.

ID	Requirement	Pri.	Use case
FR-ATT-01	A gym day shall be a day with at least one logged set or a manual check-in.	M	UC-15, UC-16
FR-ATT-02	The system shall show a Saturday-to-Friday row marking each day as Done, Missed, Today or upcoming.	M	UC-15
FR-ATT-03	The user shall be able to check a day in and undo it, except days that are done through logged sets, and not for future days.	M	UC-16
FR-ATT-04	The weekly goal shall equal the number of non-rest plan days with exercises; with no plan it shall fall back to the user's default gym days per week (3 by default).	M	UC-15
FR-ATT-05	The system shall show the gym days this week as a count out of the weekly goal.	M	UC-15
FR-ATT-06	The week streak shall count consecutive weeks that reached the weekly goal.	M	UC-15

4.8 C8 Body Tracking

Weigh-ins and weight-goal progress.

ID	Requirement	Pri.	Use case
FR-BODY-01	The user shall be able to set a start weight, target weight and optional target date.	M	UC-17
FR-BODY-02	The user shall be able to add and delete dated weigh-ins; dates in the future are not allowed.	M	UC-17
FR-BODY-03	The system shall compute progress as (start - current) / (start - target), limited to 0-100%, for both weight loss and gain.	M	UC-17, UC-15
FR-BODY-04	The system shall show progress on a barbell-style progress bar on Home and a weight chart on the body page.	M	UC-15, UC-17

4.9 C9 InBody Reports

Body-composition reports, comparison, image compression.

ID	Requirement	Pri.	Use case
FR-INB-01	The user shall be able to record a report with date, weight, skeletal muscle, fat mass, fat %, visceral level, BMR, notes and an optional image.	M	UC-18
FR-INB-02	The system shall compress the image in the browser and store it inside the report document (no separate file storage).	M	UC-18
FR-INB-03	The user shall be able to view the report image and delete a report.	S	UC-18
FR-INB-04	The system shall compare any two reports metric by metric, indicating whether the change is good or bad for that metric.	S	UC-19
FR-INB-05	Home shall show the change in muscle and body fat since the last report.	S	UC-15

4.10 C10 Units & Dates (Shared Kernel)

kg/lb conversion, rounding, date strings, Saturday-first week math.

ID	Requirement	Pri.	Use case
FR-UNT-01	All weights shall be stored in kg and converted for display and input according to the user's unit.	M	All
FR-UNT-02	All dates shall be stored as YYYY-MM-DD text.	M	All
FR-UNT-03	Weeks shall start on Saturday everywhere (plan, check-in row, streak, charts).	M	All

4.11 C11 UI Kit

Reusable interface parts and theme (Sheet, Modal, Stepper, Undo toast, PlateBar, chart card).

ID	Requirement	Pri.	Use case
FR-UI-01	The system shall provide a bottom sheet (full-height on phones, centred panel on desktop) that closes with a close button, Escape, a tap outside, or a swipe down.	M	UC-06, UC-13
FR-UI-02	Bottom sheets shall trap keyboard focus while open and return focus to the element that opened them.	M	All
FR-UI-05	Modal dialogs shall use dialog semantics and a labelled close button.	S	All
FR-UI-03	The system shall provide a numeric stepper, a modal, an undo toast and a barbell progress bar as reusable parts.	M	All
FR-UI-04	The interface shall follow the design system: slate base, ember #FF6A2B for effort, volt #43D9F0 for data, gold #FFC53D for personal records, Big Shoulders Display and Figtree fonts.	M	All

4.12 C12 App Shell & Composition Views

Routing, navigation and pages that combine components without business logic.

ID	Requirement	Pri.	Use case
FR-APP-01	The system shall provide navigation to Home, Lifts, Plan, Body, InBody, Charts and Settings: a bottom tab bar below 860 px width and a side/top bar above.	M	All
FR-APP-02	Home shall combine: weight progress, gym days this week, week streak, the check-in row, today's planned exercises, recent PRs and the latest InBody summary.	M	UC-15
FR-APP-03	Home "Today" shall list today's planned exercises with a sets-done counter and open the exercise sheet on the Log tab; it shall show "Rest day" or "No plan yet" when appropriate and offer "Do something else today".	M	UC-15, UC-13
FR-APP-04	The exercise sheet shall provide three tabs: Progress, How to and Log.	M	UC-06, UC-13, UC-14

5. External Interface Requirements

5.1 User interfaces

- Mobile-first layout with a bottom tab bar under 860 px and a navigation bar on larger screens.
- Main screens: Sign in, Home, Lifts (library), Exercise page, Plan, My body, InBody, Charts, Settings, plus the exercise sheet and picker sheets.

- Dark slate theme with ember, volt and gold accents; headings in Big Shoulders Display, body text in Figtree.
- Feedback for every action: saved states, error messages, and an Undo toast for plan changes.

5.2 Hardware interfaces

None beyond a device with a browser. Optional: camera or photo library to choose an InBody image.

5.3 Software interfaces

System	Used for	Notes
Firebase Authentication	Email/password and Google sign-in	Accessed only by the Identity component.
Cloud Firestore	All persistent data	Accessed only by the Data Store component; live subscriptions.
Image CDN	Exercise start/end images	jsDelivr first, GitHub raw as fallback.
YouTube	How-to videos	Opened as a search link in a new tab; nothing embedded or stored.
Vercel	Hosting and deployment	Deploys every push to main; previous deployments can be restored.

5.4 Communication interfaces

All traffic uses HTTPS. Firestore uses its client SDK channel; no custom server endpoints exist.

6. Non-Functional Requirements

6.1 Performance

ID	Requirement
NFR-PERF-01	The first screen after sign-in shall be interactive within 3 seconds on a mid-range phone over a 4G connection; the exercise library (about 600 KB) shall load on demand in a separate bundle.
NFR-PERF-02	Logging a set shall show feedback within 100 ms of the tap (updates are applied to the screen before the server confirms).

6.2 Security & privacy

ID	Requirement
NFR-SEC-01	Each user shall be able to read and write only the documents under their own users/{uid} path, enforced by Firestore security rules.
NFR-SEC-02	Firebase keys shall be supplied through environment variables and shall not be committed to the repository.
NFR-SEC-03	The app shall not collect payment details or third-party analytics data.

6.3 Usability & accessibility

ID	Requirement
NFR-USE-01	The interface shall be mobile-first; every tap target shall be at least 44 x 44 px.
NFR-USE-02	Every icon-only button shall have an accessible name; dialogs shall use dialog semantics and be fully usable with a keyboard.
NFR-USE-03	A user shall be able to log a planned set from Home in no more than three taps after opening the app.
NFR-USE-04	Destructive plan changes shall be undoable instead of asking for confirmation.

6.4 Reliability & data integrity

ID	Requirement
NFR-REL-01	No plan operation shall delete logs, exercises or the legacy schedule.
NFR-REL-02	If a save fails, the system shall show an error and shall leave the stored data unchanged.
NFR-REL-03	Creating a library exercise document shall be idempotent (repeating it shall not overwrite existing data).

6.5 Maintainability

ID	Requirement
NFR-MNT-01	The code shall be organised as the components in Section 3 with one owner per Firestore collection and no circular dependencies.
NFR-MNT-02	Business rules (plan editing, statistics, PR detection) shall be pure functions covered by automated unit tests.
NFR-MNT-03	The production build (npm run build) and the tests (npm test) shall pass before any change is released.

6.6 Portability & compatibility

ID	Requirement
NFR-POR-01	The app shall run in current versions of Chrome, Safari, Firefox and Edge on phones and desktops.
NFR-POR-02	The only code that depends on Firebase shall be inside Data Store and Identity, so the backend can be replaced.

6.7 Cost & deployment

ID	Requirement
NFR-COST-01	The system shall run within the free tiers of Firebase (Spark plan), Vercel and the image CDN; no paid service and no Firebase Storage shall be required.
NFR-COST-02	Every push to the main branch shall deploy automatically; any earlier deployment shall be restorable from the hosting dashboard.

7. Data Requirements

All data lives under users/{uid}. Each collection is owned by one component; other components use that component's interface.

Collection	Owner	Main fields
users/{uid}	C1	name, unit (kg|lb), startWeight, targetWeight, targetDate, weeklyGoal, planMigrated, hideUnplaced
users/{uid}/weights/{id}	C8	value (kg), date
users/{uid}/exercises/{id}	C3	name, archived, favorite, libId (library) or custom, folderKey, equipment (custom)
users/{uid}/folders/{id}	C3	name, exerciseIds[]
users/{uid}/logs/{id}	C4	exerciseId, exerciseName, weight (kg), reps, sets, date
users/{uid}/plan/{sat..fri}	C6	label, rest, focus[], items[{exerciseId, targetSets, targetReps, note}]
users/{uid}/schedule/{id}	C6 (legacy, read-only)	name, weekday (0-6 or null), exerciseIds[]
users/{uid}/checkins/{date}	C7	date (document id = date; existence means gym day)
users/{uid}/inbody/{id}	C9	date, weight, muscle, fatMass, fatPct, visceral, bmr, note, image (compressed data URL)

Library exercise documents use the deterministic id lib_<datasetId>. Plan documents use the ids sat, sun, mon, tue, wed, thu, fri. Check-in documents use the date as id. The Firestore rule users/{uid}/{document=**} already covers every collection above.

8. Business Rules

- BR-01: Weights are stored in kg and converted only for display and input.
- BR-02: Dates are YYYY-MM-DD strings; the week starts on Saturday; stored weekday numbers use JavaScript getDay() (0 = Sunday).
- BR-03: Progress % = (start - current) / (start - target), clamped to 0-100.
- BR-04: A PR is a set whose weight beats every earlier set of that exercise.
- BR-05: A gym day is a day with a logged set or a manual check-in.
- BR-06: Weekly goal = number of non-rest plan days with exercises; otherwise the default gym days per week (3).
- BR-07: Week streak = consecutive weeks that reached the weekly goal.
- BR-08: Deleting a folder or a plan day never deletes exercises or logs.
- BR-09: Trend compares the latest session's top weight with the average of up to three earlier sessions; under 2% change is steady.

9. Future Requirements

ID	Feature	Outline
F1	Crew and leaderboard (C13)	Opt-in sharing, fair ranking by percentage improvement, in a new shared collection; reads progress through IProgress.
F2	Challenges and badges	Goals and rewards built on attendance and logging.
F3	Progress photos	Photos without paid storage.
F4	Arabic / RTL	Translated interface with right-to-left layout.

Appendix A. Use Case List

Actors: Athlete (primary); Firebase Auth/Google, Firestore, Image CDN and YouTube (supporting systems).

ID	Use case	Actor	Requirements
UC-01	Register account	Athlete	FR-ID-01, FR-ID-05
UC-02	Sign in (email or Google)	Athlete	FR-ID-02, FR-ID-03, FR-ID-05, FR-ID-07
UC-03	Sign out	Athlete	FR-ID-04
UC-04	Edit profile and settings	Athlete	FR-ID-06
UC-05	Browse and search exercises	Athlete	FR-LIB-01, FR-LIB-02, FR-LIB-03, FR-LIB-04, FR-LIB-12
UC-06	View exercise details and how-to	Athlete	FR-LIB-05, FR-LIB-06, FR-LIB-07, FR-UI-01, FR-APP-04
UC-07	Manage favorites	Athlete	FR-LIB-08, FR-LIB-09
UC-08	Manage custom exercises	Athlete	FR-LIB-10
UC-09	Manage exercise folders	Athlete	FR-LIB-11
UC-10	Build the weekly plan	Athlete	FR-LIB-09, FR-PLN-01, FR-PLN-02, FR-PLN-03, FR-PLN-04, FR-PLN-05, FR-PLN-11
UC-11	Edit the week (shift, move, swap, copy, clear, undo)	Athlete	FR-PLN-07, FR-PLN-08, FR-PLN-09, FR-PLN-10
UC-12	Apply a starter plan	Athlete	FR-PLN-06
UC-13	Log a set	Athlete	FR-LIB-09, FR-LOG-01, FR-LOG-02, FR-LOG-03, FR-LOG-04, FR-LOG-05, FR-LOG-06, FR-LOG-07, FR-LOG-08, FR-LOG-09, FR-UI-01, FR-APP-03, FR-APP-04
UC-14	Review exercise progress	Athlete	FR-PRG-01, FR-PRG-02, FR-PRG-03, FR-PRG-04, FR-PRG-06, FR-PLN-13, FR-APP-04
UC-15	View Home dashboard	Athlete	FR-PRG-04, FR-ATT-01, FR-ATT-02, FR-ATT-04, FR-ATT-05, FR-ATT-06, FR-BODY-03, FR-BODY-04, FR-INB-05, FR-APP-02, FR-APP-03
UC-16	Check in or undo a gym day	Athlete	FR-ATT-01, FR-ATT-03
UC-17	Set weight goal and log weigh-ins	Athlete	FR-BODY-01, FR-BODY-02, FR-BODY-03, FR-BODY-04
UC-18	Record an InBody report	Athlete	FR-INB-01, FR-INB-02, FR-INB-03
UC-19	Compare InBody reports	Athlete	FR-INB-04
UC-20	View charts	Athlete	FR-PRG-05
UC-21	Add an exercise to today's plan	Athlete	FR-PLN-12

Appendix B. Requirement to Component Map

ID	Component	Requirement groups
C1	Identity & Profile	FR-ID (7 requirements)
C2	Data Store	FR-DS (4 requirements)
C3	Exercise Library	FR-LIB (12 requirements)
C4	Workout Logging	FR-LOG (9 requirements)
C5	Progress & Analytics	FR-PRG (6 requirements)
C6	Weekly Plan	FR-PLN (13 requirements)
C7	Attendance & Goals	FR-ATT (6 requirements)
C8	Body Tracking	FR-BODY (4 requirements)
C9	InBody Reports	FR-INB (5 requirements)
C10	Units & Dates (Shared Kernel)	FR-UNT (3 requirements)
C11	UI Kit	FR-UI (5 requirements)
C12	App Shell & Composition Views	FR-APP (4 requirements)

Part 2. Component Specification

What each component provides, requires and contains.

1. Overview

Forge is split into 13 components. Rules: (R1) one responsibility per component; (R2) one owner for every Firestore collection, other components use its interface; (R3) dependencies have no cycles; (R4) pages that combine components (Composition Views in C12) contain no business rules; (R5) only C2 (and the sign-in calls in C1) touch Firebase.

1.1 Provides and requires at a glance

Component	Provides	Requires (components / systems)
C1 Identity & Profile	IAuth, IProfile	C2, Firebase Auth SDK
C2 Data Store	IDataStore	Firestore SDK, Current uid
C3 Exercise Library	IExerciseCatalog, IExerciseSearch, Static data	C2, C10, Image CDN, YouTube
C4 Workout Logging	ILogger, ILogReader	C2, C3, C5, C10
C5 Progress & Analytics	IProgress	C10, C11
C6 Weekly Plan	IPlan	C2, C3, C5, C1, C11
C7 Attendance & Goals	IAttendance	C2, C4, C6, C1, C10
C8 Body Tracking	IBodyTracker, IBodyReader	C2, C1, C10, C11
C9 InBody Reports	IInBody, IInBodyReader	C2, C10, C11
C10 Units & Dates (Shared Kernel)	IUnits, IDates	None
C11 UI Kit	UI components, Theme	Third-party libraries
C12 App Shell & Composition Views	Routes, Views	C1, C3, C4, C5, C6, C7, C8, C9, C11, C10
C13 Crew & Leaderboard	ICrew	C5, C8, C1, C2

1.2 Interface catalogue

Interface	Provided by	Required by
IAuth	C1 Identity & Profile	C12
IProfile	C1 Identity & Profile	C6, C7, C8, C12, C13
IDataStore	C2 Data Store	C1, C3, C4, C6, C7, C8, C9, C13
IExerciseCatalog	C3 Exercise Library	C4, C6, C12
IExerciseSearch	C3 Exercise Library	C6, C12
ILogger	C4 Workout Logging	C12
ILogReader	C4 Workout Logging	C7, C12
IProgress	C5 Progress & Analytics	C4, C6, C12, C13
IPlan	C6 Weekly Plan	C7, C12
IAttendance	C7 Attendance & Goals	C12
IBodyTracker	C8 Body Tracking	C12
IBodyReader	C8 Body Tracking	C12, C13
IInBody	C9 InBody Reports	C12
IInBodyReader	C9 InBody Reports	C12
IUnits	C10 Units & Dates (Shared Kernel)	C4, C5, C8, C9, C12
IDates	C10 Units & Dates (Shared Kernel)	C3, C4, C5, C7, C12
ICrew	C13 Crew & Leaderboard	-

1.3 Collection ownership

Collection	Owner	Other components use it through
users/{uid}	C1	IProfile (C6, C7, C8, C12)
exercises, folders	C3	IExerciseCatalog (C4, C6, C12)
logs	C4	ILogReader (C7, C12); passed as arguments to C5 and C6 rows
plan, schedule (legacy)	C6	IPlan (C7, C12)
checkins	C7	IAttendance (C12)
weights	C8	IBodyReader (C12)
inbody	C9	IInBodyReader (C12)

2. Component Details

2.1 C1 Identity & Profile

Item	Detail
Type	Feature component
Purpose	Knows who the athlete is. Handles account creation, sign in (email and Google), sign out, the session, and the profile document (name, unit, goals).
Requirements covered	FR-ID-01 to FR-ID-07

Responsibilities

- Create an account with name, email and password.
- Sign in with email/password or a Google pop-up; translate error codes into friendly messages.
- Keep the session and expose the current user (loading, signed out, signed in).
- Create the default profile on the first sign-in.
- Read and update the profile (name, unit kg/lb, start/target weight, target date, default gym days per week, migration flags).

Provided interfaces

Interface	Operation	Description
IAuth	user	Current Firebase user: undefined while loading, null when signed out.
IAuth	signUp(name, email, password)	createUserWithEmailAndPassword, then updateProfile(displayName) when a name was typed.
IAuth	signIn(email, password)	signInWithEmailAndPassword.
IAuth	signInWithGoogle()	signInWithPopup with the Google provider.
IAuth	signOut()	Ends the session; profile and data are cleared from memory.
IProfile	profile	Live profile document: name, unit, startWeight, targetWeight, targetDate, weeklyGoal, planMigrated, hideUnplaced.
IProfile	saveProfile(partial)	Merges the given fields into users/{uid} (does not overwrite the others).
IProfile	ensureProfile()	If users/{uid} does not exist, creates it with name (display name or email prefix), unit "kg", empty goal.

Required interfaces

Needs	From	Operations used	Why
IDataStore	C2 Data Store	merge on users/{uid}, live read of users/{uid}	Stores and watches the profile document.
Firebase Auth SDK	External	onAuthStateChanged, create/sign-in/sign-out calls	Authentication itself.

Data owned

- users/{uid}: name, unit, startWeight, targetWeight, targetDate, weeklyGoal, planMigrated, hideUnplaced, createdAt

Where it lives in the code today

- pages/Login.jsx (sign in/up UI and error mapping)
- AppContext.jsx (auth state + profile creation; to be split)
- firebase.js (auth, googleProvider)

Rules and behaviour

- The uid is the root of all data; no other component reads another user's data.
- Units are a profile field; changing it never changes stored data (everything is kg).
- Weekly goal choices are 1 to 7 days and are used only until a plan exists.

Known gaps against the target design

- Auth and profile code lives inside AppContext.jsx next to the data helpers and unit helpers; to be split into C1, C2 and C10.

2.2 C2 Data Store

Item	Detail
Type	Foundation component
Purpose	The only place that talks to Cloud Firestore. Gives every other component a small, uniform way to read live collections and write documents under users/{uid}.
Requirements covered	FR-DS-01 to FR-DS-04

Responsibilities

- Provide live, auto-updating lists of documents for any collection under the user.
- Create, replace, merge, update and delete documents by id.
- Create a document only if it does not exist (idempotent).
- Stamp created documents with a server timestamp.

Provided interfaces

Interface	Operation	Description
IDataStore	useCol(name)	Live list { items, loading } of users/{uid}/{name}; each item includes its id.
IDataStore	add(name, data)	addDoc with auto id and createdAt (serverTimestamp).
IDataStore	put(name, id, data)	setDoc with a fixed id and createdAt; replaces the document.
IDataStore	merge(name, id, data)	setDoc with merge: creates the document if missing, otherwise updates only the given fields (arrays are replaced whole).

Interface	Operation	Description
IDataStore	ensureDoc(name, id, data)	Creates the document only if it is missing; never overwrites; returns the id.
IDataStore	upd(name, id, data)	updateDoc (document must exist).
IDataStore	del(name, id)	deleteDoc.

Required interfaces

Needs	From	Operations used	Why
Firestore SDK	External	collection, doc, onSnapshot, addDoc, setDoc, updateDoc, deleteDoc, getDoc	Persistence.
Current uid	Injected by C12	user.uid at wiring time	Builds the users/{uid}/... path. It is passed in by the shell, so C2 has no compile-time dependency on C1 (this avoids a C1 <-> C2 cycle).

Data owned

- None. C2 stores nothing of its own; it is the gateway for every collection.

Where it lives in the code today

- AppContext.jsx (add/put/merge/ensureDoc/upd/del and useCol; to be extracted)
- firebase.js (db)

Rules and behaviour

- No component other than C2 (and the Auth call in C1) imports the Firebase SDK.
- A failed write rejects the returned promise; the caller decides how to show the error.

Known gaps against the target design

- Today useCol, add, put... live in AppContext.jsx together with auth, profile and units.

2.3 C3 Exercise Library

Item	Detail
Type	Feature component
Purpose	Everything about exercises as things: the 584 built-in exercises, how to find them, how they look, and the athlete's own exercise documents (favorites, custom exercises, folders).
Requirements covered	FR-LIB-01 to FR-LIB-12

Responsibilities

- Load the exercise dataset lazily (separate bundle) and build a search index.
- Browse by 17 muscle folders plus "My custom"; search with synonyms; filter by equipment and popular.
- Show muscles, equipment, mechanic, start/end images (CDN with fallback) and steps; offer a YouTube "Watch how-to" link.
- Favorites; create, rename, archive/restore and delete custom exercises; user folders (create, rename, delete, add/remove exercises).
- Create the personal document of a library exercise only on first use, with the deterministic id lib_<datasetId>.

Provided interfaces

Interface	Operation	Description
IExerciseCatalog	useExerciseLibrary()	Returns { data, index, byId, error }; data is null until the dataset has loaded.
IExerciseCatalog	loadExercises()	Dynamic import of exercises.json (cached promise).
IExerciseCatalog	libDocId(id) / datasetIdOf(docId)	Convert between dataset id and the document id lib_<id>.
IExerciseCatalog	ensureLibExercise(app, libEx)	Idempotent creation of exercises/lib_<id> { name, libId, favorite false, archived false }.
IExerciseCatalog	imageUrls(ex, i)	Returns [jsDelivr URL, raw.githubusercontent URL] for image i.
IExerciseCatalog	libView / customView / docView(doc, byId)	Uniform view model { key, docId, name, lib, group, mechanic, folder, popular, custom, doc }.
IExerciseCatalog	favorites, custom exercises, folders	merge("exercises", id, {favorite}); add("exercises", {name, archived, favorite, custom: true, folderKey, equipment}); upd archived/name; add/upd/del "folders" {name, exerciseIds}.
IExerciseSearch	buildIndex(data, popularSet)	Builds the searchable index (popular flag included).
IExerciseSearch	searchIndex(index, query, {folder, equipment, popularOnly, limit})	Ranked matches; popular exercises get a bonus; synonyms expanded.
IExerciseSearch	folderItems(index, folder, opts)	Exercises of one muscle folder, popular first then A-Z.
IExerciseSearch	expandQuery(query), tokens(text)	Query normalisation and synonym expansion.
Static data	FOLDERS, CUSTOM_FOLDER, EQUIPMENT_GROUPS, MECHANIC_LABEL, POPULAR_IDS, howToUrl(name)	17 muscle folders with emoji; 6 equipment groups; 122 popular ids; YouTube search URL.

Required interfaces

Needs	From	Operations used	Why
IDataStore	C2 Data Store	useCol("exercises"), useCol("folders"), merge, add, upd, del, ensureDoc	Personal exercise documents and user folders.
IDates	C10 Units & Dates	byDate, fmtDate	Ordering and showing dates in lists.
Image CDN, YouTube	External	HTTP GET for images; open a search URL	Media and how-to links (nothing embedded or stored).

Data owned

- users/{uid}/exercises/{id}: name, archived, favorite, libId, custom, folderKey, equipment
- users/{uid}/folders/{id}: name, exerciseIds[]

Where it lives in the code today

- data/exercises.json, data/muscleFolders.js, data/popular.js, data/videos.json
- lib/exerciseData.js, exerciseSearch.js, exerciseSummary.js, exerciseViews.js (libView/customView/docView)
- pages/Exercises.jsx
- components/exercise/ExerciseImage, ExerciseMedia, MuscleChips, HowTo, FavoriteButton, MyCustom

Rules and behaviour

- Library exercises have no personal document until first favorite, log or add-to-plan.
- Deleting a folder never deletes exercises or logs.
- videos.json stays empty; the only video feature is the search link.

Known gaps against the target design

- Exercises.jsx also reads logs to show "last weight" and recent exercises, and ExerciseDetail deletes an exercise together with its logs. In the target design these two cross-component actions belong to C12 (Lifts view and Exercise detail view), so C3 does not depend on C4.
- exerciseViews.js also holds recentExerciseIds and groupLogs, which belong to C4.

2.4 C4 Workout Logging

Item	Detail
Type	Feature component
Purpose	Records what the athlete lifted. Owns the logs collection and the quick-logging behaviour (prefill, counter, PR announcement, rest timer, unsaved values).
Requirements covered	FR-LOG-01 to FR-LOG-09

Responsibilities

- Write one log document per set { exerciseId, exerciseName, weight kg, reps, sets, date }.
- Prefill weight and reps from the last session or from an unsaved draft.
- Show the sets-done counter for today against the target (for example 2/3).
- Announce a new PR when the weight beats all earlier sets of that exercise.
- Optional 90 second rest timer (choice remembered).
- History of sets with delete; detailed entry form on the exercise page.

Provided interfaces

Interface	Operation	Description
ILogger	addSet({exerciseId, exerciseName, weight, reps, sets, date})	add("logs", ...) with weight already in kg; the quick logger always writes sets = 1.
ILogger	deleteLog(id)	del("logs", id).
ILogger	getDraft(exerciseId) / setDraft(exerciseId, values)	Unsaved weight and reps kept per exercise in sessionStorage ("forge.setDrafts"); cleared after logging.
ILogger	rest timer preference	90 seconds; on/off stored in localStorage ("forge.restTimer").
ILogReader	useCol("logs")	Live list of all logs.
ILogReader	groupLogs(logs)	Map exerciseId -> logs of that exercise.
ILogReader	recentExerciseIds(logs, n = 6)	Most recently logged exercise ids, newest first.
ILogReader	setsOn(logs, date)	Number of sets logged on a date (sum of the sets field).

Required interfaces

Needs	From	Operations used	Why
IDataStore	C2 Data Store	useCol("logs"), add, del	Stores logs.
IExerciseCatalog	C3 Exercise Library	ensureLibExercise	Creates the exercise document before the first log of a library exercise.
IProgress	C5 Progress & Analytics	bestWeight, lastLog	PR check and prefill.
IUnits	C10 Units & Dates	show, toKg, unit	Display and input in kg or lb.
IDates	C10 Units & Dates	todayStr	Log date.

Data owned

- users/{uid}/logs/{id}: exerciseId, exerciseName, weight (kg), reps, sets, date, createdAt

Where it lives in the code today

- components/exercise/LogTab.jsx
- lib/setDrafts.js
- logging form and history in pages/ExerciseDetail.jsx
- groupLogs and recentExerciseIds currently in lib/exerciseViews.js

Rules and behaviour

- PR = weight greater than every earlier set of that exercise, and only if at least one earlier set exists.
- Dates cannot be in the future on the detailed form.
- Weights are converted to kg before they are written.

Known gaps against the target design

- LogTab also contains the "Add to today's plan" button which edits the plan directly; in the target design it calls C6 (IPlan.addExerciseToDay).

2.5 C5 Progress & Analytics

Item	Detail
Type	Feature component (pure functions and views)
Purpose	Turns a list of logs into meaning: best weight, trend, PRs, sessions, and the small progress views. It has no storage of its own and receives the data it analyses as arguments.
Requirements covered	FR-PRG-01 to FR-PRG-06

Responsibilities

- Compute best weight, last set, last five sets, sessions (top weight per day) and trend.
- Mark PRs.
- Show the progress panel (best weight, mini chart of the last 8 sessions, PR badge, last 5 sets) and the one-line glance (last, best, trend arrow).
- Provide estimated 1RM (Epley) for the Charts page only.

Provided interfaces

Interface	Operation	Description
IProgress	sessionsOf(logs)	One entry per training day, oldest first: { date, top, sets }.
IProgress	bestWeight(logs) / lastLog(logs) / lastSets(logs, k)	Heaviest weight, newest log, newest k logs.
IProgress	trendOf(logs)	"up" | "down" | "flat" | null: latest session top vs average of up to 3 earlier sessions; under 2% is flat; null with fewer than 2 sessions.
IProgress	computePRs(logs)	Set of log ids that are PRs.
IProgress	epley(weight, reps)	Estimated 1-rep max (weight x (1 + reps/30), for reps > 1).
IProgress	ProgressPanel, Glance, TrendArrow (views)	Presentational views that take the logs of one exercise as a prop.

Required interfaces

Needs	From	Operations used	Why
IUnits	C10 Units & Dates	show, unit	Showing weights in the user's unit.
IDates	C10 Units & Dates	fmtDate, shortDate	Labels in the views.
UI Kit	C11	chart drawing helpers (Recharts), theme colors	The mini chart.

Data owned

- None (derived data only).

Where it lives in the code today

- lib/exerciseStats.js
- computePRs and epley in utils.js
- components/exercise/ProgressPanel.jsx, Glance.jsx

Rules and behaviour

- The component never reads the database; callers pass the logs (C12 views and C6 rows pass them in).
- No estimated 1RM is shown in new screens.

Known gaps against the target design

- computePRs and epley are still in utils.js; the Charts page builds its own series in the page.

2.6 C6 Weekly Plan

Item	Detail
Type	Feature component
Purpose	The athlete's week: seven days (Saturday to Friday), each with a label, a rest flag, focus muscles and an ordered list of exercises with sets and reps targets, plus all the ways to reshape it safely.
Requirements covered	FR-PLN-01 to FR-PLN-13

Responsibilities

- Hold and edit the seven day documents; add, remove, reorder exercises; set sets and reps; toggle focus muscles and rest.
- Suggest exercises for a day (focus popular, favorites, recents, browse by muscle, search).
- Starter plans that replace the week (PPL, Upper/Lower, Full body, Bro split).
- Week editing: shift by N days (wrap-around), move, swap, copy a day, clear a day; every change undoable.
- Write only changed days; roll back the in-memory state if saving fails.
- One-time migration of the old schedule into the plan.
- Compute the weekly goal (non-rest days with exercises).

Provided interfaces

Interface	Operation	Description
IPlan	usePlan()	{ plan, loading, unplacedDays }; plan = { sat..fri: { label, rest, focus, items } }; triggers the one-time migration.
IPlan	usePlanActions()	{ plan, save, update(dayKey, fn), apply(nextPlan), addExercise(dayKey, view, sets, reps), applyTemplate(template, libData) }.
IPlan	Pure operations	addItem, removeItem, moveItem, setTargets, toggleFocus, hasItem, shiftWeek(plan, n), swapDays, moveDay, copyDay, clearDay, dayChanged, normalizeDay, planFromDocs, emptyDay.
IPlan	planGoal(plan) / planHasItems(plan)	Number of non-rest days with at least one exercise / whether any day has exercises.

Interface	Operation	Description
IPlan	DAYS, DAY_KEYS, dayKeyOf(date), dayInfo(key)	Seven days in display order with their getDay() numbers; today's key.
IPlan	migrateSchedule(scheduleDocs, existingIds)	{ plan, unplaced }: converts the legacy schedule; unscheduled days become rest days; days with no weekday are only counted.
IPlan	TEMPLATES, templateNames(t)	Four starter plans.
IPlan	Views	DayCard, PlanPicker, ShiftWeekSheet, DayPickerSheet, TemplatesModal, AddToPlanButton, TodayCard.

Required interfaces

Needs	From	Operations used	Why
IDataStore	C2 Data Store	useCol("plan"), useCol("schedule"), merge	Plan documents and the legacy schedule (read-only).
IExerciseCatalog	C3 Exercise Library	useExerciseLibrary, ensureLibExercise, docView	Resolve exercises and create their documents on first add.
IExerciseSearch	C3 Exercise Library	folderItems, searchIndex	Picker suggestions and search.
IProgress	C5 Progress & Analytics	Glance	Last/best/trend line on each row.
IProfile	C1 Identity & Profile	saveProfile({planMigrated, hideUnplaced})	Migration flags.
UI Kit	C11	Sheet, Modal, Stepper, UndoToast	Interface parts.

Data owned

- users/{uid}/plan/{sat|sun|mon|tue|wed|thu|fri}: label, rest, focus[], items[{exerciseId, targetSets, targetReps, note}]
- users/{uid}/schedule/{id} (legacy; read-only, never modified)

Where it lives in the code today

- lib/plan.js, usePlan.js, usePlanActions.js
- data/planTemplates.js
- components/plan/*
- pages/Plan.jsx

Rules and behaviour

- Weekly goal = non-rest days with exercises.
- The same exercise cannot be added twice to one day.
- Rest days keep their exercises.
- Targets: sets 1-20, reps 1-100 (defaults 3 x 10).
- Every shift/move/swap/copy/clear and exercise removal offers Undo for 8 seconds.

Known gaps against the target design

- LogTab edits the plan directly for "Add to today's plan"; TodayCard (C12 composition) lives in components/plan.
- Plan.jsx holds the undo orchestration; it can move into usePlanActions.

2.7 C7 Attendance & Goals

Item	Detail
Type	Feature component
Purpose	Answers "did I show up?": which days count as gym days, the weekly goal and the week streak, and owns manual check-ins.
Requirements covered	FR-ATT-01 to FR-ATT-06

Responsibilities

- Store manual check-ins (a document per date).
- Define a gym day = a day with a logged set or a check-in.
- Compute the weekly goal: planGoal(plan), or the profile's default gym days (3) when there is no plan.
- Compute days done this week and the week streak (consecutive weeks reaching the goal).
- Mark each day of the Saturday-Friday week as Done, Missed, Today or upcoming; allow undoing a check-in (not for days done through logged sets, not for future days).

Provided interfaces

Interface	Operation	Description
IAttendance	useCol("checkins")	Live list; the document id is the date.
IAttendance	checkIn(date) / undoCheckIn(date)	put("checkins", date, {date}) / del("checkins", date).
IAttendance	weekStreak(dates, goal)	The current week counts if it already reached the goal; then each earlier consecutive week that reached it. A week reaches the goal with at least goal distinct training days.
IAttendance	weekly goal rule	planGoal(plan) || profile.weeklyGoal ?? 3 (WEEK_GOAL).
IAttendance	week row states	done / missed / today / future per day of this week.

Required interfaces

Needs	From	Operations used	Why
IDataStore	C2 Data Store	useCol("checkins"), put, del	Check-ins.
ILogReader	C4 Workout Logging	dates of logged sets	A logged set makes the day a gym day.
IPlan	C6 Weekly Plan	planGoal	Weekly goal.
IProfile	C1 Identity & Profile	weeklyGoal	Fallback goal.
IDates	C10 Units & Dates	weekStart, todayStr, ymd	Week boundaries (Saturday start).

Data owned

- users/{uid}/checkins/{date}: date, createdAt (existence = gym day)

Where it lives in the code today

- weekStreak and WEEK_GOAL in utils.js
- check-in logic and week row inside pages/Dashboard.jsx
- goal field UI in pages/Settings.jsx

Rules and behaviour

- Weeks start on Saturday.
- A day with logged sets cannot be un-checked (it stays done).

Known gaps against the target design

- The attendance logic is spread across utils.js and Dashboard.jsx.

2.8 C8 Body Tracking

Item	Detail
Type	Feature component
Purpose	Body weight over time and progress toward the weight goal.
Requirements covered	FR-BODY-01 to FR-BODY-04

Responsibilities

- Add and delete dated weigh-ins (not in the future); the first weigh-in also becomes the start weight when none is set.
- Save the goal: start weight, target weight, optional target date.
- Compute progress % = (start - current) / (start - target), clamped 0-100, for loss and gain.
- Show a weight chart and the barbell progress bar.

Provided interfaces

Interface	Operation	Description
IBodyTracker	addWeighIn(value, date)	add("weights", {value in kg, date}); sets startWeight when it is empty.
IBodyTracker	deleteWeighIn(id)	del("weights", id).
IBodyTracker	saveGoal(start, target, date)	saveProfile({startWeight, targetWeight, targetDate}).
IBodyTracker	progressPct(start, current, target)	Percentage clamped to 0-100.
IBodyReader	useCol("weights")	Live weigh-ins (for Home and Charts).

Required interfaces

Needs	From	Operations used	Why
IDataStore	C2 Data Store	useCol("weights"), add, del	Weigh-ins.
IProfile	C1 Identity & Profile	profile goal fields, saveProfile	Goal values.
IUnits	C10 Units & Dates	show, toKg, unit	kg/lb conversion.
UI Kit	C11	PlateBar, ChartCard	Progress bar and chart.

Data owned

- users/{uid}/weights/{id}: value (kg), date, createdAt

Where it lives in the code today

- pages/Body.jsx
- progressPct in utils.js
- components/PlateBar.jsx (UI Kit)

Rules and behaviour

- Weights are stored in kg.
- Progress works for both losing and gaining weight.

Known gaps against the target design

- progressPct lives in utils.js.

2.9 C9 InBody Reports

Item	Detail
Type	Feature component
Purpose	Body-composition scan reports: record, view, compare and chart them.
Requirements covered	FR-INB-01 to FR-INB-05

Responsibilities

- Record a report: date, weight, skeletal muscle, fat mass, fat %, visceral level, BMR, notes, optional image.
- Compress the image in the browser and store it inside the document.
- Compare any two reports metric by metric with good/bad direction per metric.
- View the image, delete a report; supply metric series for charts and the Home summary.

Provided interfaces

Interface	Operation	Description
IInBody	addReport(data)	add("inbody", {date, note, image, metrics}); mass metrics converted with toKg, empty fields stored as null.
IInBody	deleteReport(id)	del("inbody", id).
IInBody	compare(reportA, reportB)	Per-metric difference with goodUp rules (muscle up is good, fat down is good).
IInBody	compressImage(file, max = 1200, quality = 0.74)	Scales the longest side to 1200 px and returns a JPEG data URL.
IInBody	IN_METRICS	weight, muscle, fatMass, fatPct, visceral, bmr with kind and goodUp.
IInBodyReader	useCol("inbody")	Live reports (for Home and Charts).

Required interfaces

Needs	From	Operations used	Why
IDataStore	C2 Data Store	useCol("inbody"), add, del	Reports.
IUnits	C10 Units & Dates	show, toKg	Mass metrics in kg/lb.
UI Kit	C11	Modal, ChartCard	Form and charts.

Data owned

- users/{uid}/inbody/{id}: date, weight, muscle, fatMass, fatPct, visceral, bmr, note, image (data URL), createdAt

Where it lives in the code today

- pages/InBody.jsx
- compressImage and IN_METRICS in utils.js

Rules and behaviour

- No Firebase Storage is used (free plan); the image is part of the report document.
- Dates cannot be in the future.

Known gaps against the target design

- compressImage and IN_METRICS live in utils.js.

2.10 C10 Units & Dates (Shared Kernel)

Item	Detail
Type	Foundation component
Purpose	Small, dependency-free rules every component shares: kilograms vs pounds, rounding, date strings and the Saturday-first week.
Requirements covered	FR-UNT-01 to FR-UNT-03

Responsibilities

- Convert between kg and lb for display and input; keep storage in kg.
- Format and parse dates (YYYY-MM-DD, short and long labels).
- Week math: Saturday as the first day, display order of weekdays.

Provided interfaces

Interface	Operation	Description
IUnits	show(kg)	Value in the user's unit, rounded to 1 decimal.
IUnits	toKg(value)	Parses input in the user's unit and returns kg rounded to 2 decimals (null if not a number).
IUnits	r1(n), KG_PER_LB	Rounding helper; 0.45359237.
IDates	ymd(date), todayStr()	Date to YYYY-MM-DD; today.
IDates	fmtDate(s, opts), shortDate(s)	Readable date labels.
IDates	weekStart(s)	The Saturday that starts the week of the given date.
IDates	byDate(a, b)	Comparator by date then by creation time.
IDates	DAY_NAMES, WEEK_ORDER	Names indexed by JS getDay(); display order [6,0,1,2,3,4,5].

Required interfaces

Needs	From	Operations used	Why
None	-	-	Pure functions with no outside dependencies (the unit setting is passed in or read from IProfile by the caller).

Data owned

- None.

Where it lives in the code today

- utils.js (shared part)
- show and toKg currently created inside AppContext.jsx

Rules and behaviour

- Everything is stored in kg; dates are YYYY-MM-DD text; weeks start on Saturday.

Known gaps against the target design

- utils.js also contains logic for C5, C7, C8, C9 and C11; to be split.

2.11 C11 UI Kit

Item	Detail
Type	Foundation component
Purpose	The design system as reusable parts. It knows nothing about exercises, plans or weights.
Requirements covered	FR-UI-01 to FR-UI-04

Responsibilities

- Bottom sheet that is full-height on phones and a centred panel on desktop; closes with the button, Escape, backdrop tap or swipe down; traps focus and returns it to the opener.
- Modal dialog, numeric stepper (44 px buttons, typed input clamped on blur), undo toast (8 seconds), barbell progress bar, chart card (bar/line, zoom).
- Theme: slate base, ember #FF6A2B, volt #43D9F0, gold #FFC53D; Big Shoulders Display and Figtree; 44 px tap targets.

Provided interfaces

Interface	Operation	Description
UI components	Sheet({title, onClose, headerExtra, children})	Bottom sheet with focus trap and swipe to close.
UI components	Modal({title, onClose, wide, children})	Centered dialog.
UI components	Stepper({label, value, onChange, min, max, step, decimals, suffix})	Minus / number / plus control.
UI components	UndoToast({toast, onDismiss})	Message with Undo; toast = { id, text, onUndo }.
UI components	PlateBar({pct, start, current, target, unit})	Barbell-style progress bar.
UI components	ChartCard({title, subtitle, data, series, defaultType, defaultZoom, extra})	Chart with toggles.
UI components	useFocusTrap(ref, onEscape)	Focus cycling, Escape, scroll lock, focus return.
Theme	styles.css, COLORS	CSS variables and chart colors.

Required interfaces

Needs	From	Operations used	Why
Third-party libraries	External	React, Recharts, lucide-react icons	Rendering, charts and icons.

Data owned

- None.

Where it lives in the code today

- components/Sheet.jsx, Modal.jsx, Stepper.jsx, PlateBar.jsx, ChartCard.jsx
- components/plan/UndoToast.jsx
- lib/useFocusTrap.js, styles.css, COLORS in utils.js

Rules and behaviour

- No business logic and no data access.
- Tap targets are at least 44 px.

Known gaps against the target design

- Modal does not trap focus yet (only Sheet does); it should reuse useFocusTrap.
- UndoToast and COLORS are stored outside the UI Kit folders.

2.12 C12 App Shell & Composition Views

Item	Detail
Type	Top-level component
Purpose	Wires everything together: routing, navigation and the pages that combine several components. It contains no business rules.
Requirements covered	FR-APP-01 to FR-APP-04

Responsibilities

- Gate the app: splash while loading, Login when signed out, otherwise the layout and routes.
- Navigation: Home, Body, Lifts, Plan, InBody, Charts, Settings (bottom tab bar under 860 px, side bar above).
- Composition views: Home (Dashboard + TodayCard), Exercise detail, Exercise sheet (Progress / How to / Log), Charts, Settings, Lifts page.
- Provide the user id to the Data Store and connect interfaces between components.

Provided interfaces

Interface	Operation	Description
Routes	/ /body /exercises /exercises/:id /schedule /inbody /charts /settings	Unknown paths redirect to Home.
Views	Dashboard, ExerciseDetail, ExerciseSheet (+ useExerciseSheet), Charts, Settings, Layout, TodayCard	Each view only calls other components' interfaces.

Required interfaces

Needs	From	Operations used	Why
IAuth, IProfile	C1	user, profile, signOut, saveProfile	Gate and settings.

Needs	From	Operations used	Why
IExerciseCatalog, IExerciseSearch	C3	library and search	Lifts view, exercise sheet.
ILogger, ILogReader	C4	logging and logs	Sheet Log tab, Home counters, exercise detail.
IProgress	C5	stats and views	Progress tab, glance, Charts.
IPlan	C6	plan and views	Plan page and Today.
IAttendance	C7	check-ins, goal, streak	Home.
IBodyTracker / IBodyReader, IInBody / IInBodyReader	C8, C9	weights and reports	Body, InBody, Home, Charts.
UI Kit, IUnits, IDates	C11, C10	components and helpers	Rendering.

Data owned

- None.

Where it lives in the code today

- App.jsx, main.jsx, components/Layout.jsx
- pages/Dashboard.jsx, ExerciseDetail.jsx, Charts.jsx, Settings.jsx
- components/exercise/ExerciseSheet.jsx, useExerciseSheet.jsx, components/plan/TodayCard.jsx

Rules and behaviour

- No view duplicates a rule that belongs to a component.
- Cross-component actions (for example deleting an exercise together with its logs) are orchestrated here.

Known gaps against the target design

- Dashboard.jsx still computes goal, streak and the week row itself (C7 logic).

2.13 C13 Crew & Leaderboard

Item	Detail
Type	Planned component (roadmap)
Purpose	Opt-in sharing among friends with a fair ranking by percentage improvement. Not built yet.
Requirements covered	Section 9 of the SRS (future)

Responsibilities

- Let an athlete opt in and join a crew.
- Publish selected progress numbers to a shared collection.
- Rank crew members by percentage improvement.

Provided interfaces

Interface	Operation	Description
ICrew	join(crew), leave(crew), leaderboard()	Planned operations; details to be specified before implementation.

Required interfaces

Needs	From	Operations used	Why
IProgress	C5	bestWeight, trendOf, PRs	Numbers to share.
IBodyReader	C8	weigh-ins	Percentage improvement.
IProfile	C1	name	Display name.
IDataStore	C2	a new shared collection	Shared data (needs new security rules).

Data owned

- New shared collection (not under users/{uid}); requires new Firestore rules published in the console.

Where it lives in the code today

- None yet.

Rules and behaviour

- Strictly opt-in; private data never leaves a user's subtree without consent.

Known gaps against the target design

- Not implemented.

Part 3. Component Breakdown and Ownership

Which component owns each file, collection and shared helper.

Target architecture. Every source file, every Firestore collection and every shared helper is owned by exactly one component. Rules: (R1) one responsibility per component, (R2) one owner per collection, others read/write through the owner's interface, (R3) dependencies are acyclic, (R4) pages that combine several components are Composition Views and contain no business logic.

Components

ID	Component	Responsibility	Owns (collections)	Provided interfaces	Required interfaces
C1	Identity & Profile	Sign in/out, session, profile (name, unit, weight goal, weeklyGoal)	`users/{uid}`	IAuth, IProfile	IDataStore
C2	Data Store	The only code that talks to Firestore; live subscriptions	none	IDataStore	(Firebase SDK)
C3	Exercise Library	Dataset, search, muscle folders, media, favorites, custom exercises, user folders	`exercises`, `folders`	IExerciseCatalog, IExerciseSearch	IDataStore, IDates
C4	Workout Logging	Log sets, sets-done counter, rest timer, drafts, history	`logs`	ILogger, ILogReader	IDataStore, IExerciseCatalog, IUnits, IDates
C5	Progress & Analytics	PRs, trends, best weight, progress views	none (derived)	IProgress	IUnits, IDates (pure functions: callers pass the logs in)
C6	Weekly Plan	7-day plan, templates, shift/move/swap/copy/clear, undo, legacy schedule migration	`plan`, `schedule` (legacy, read-only)	IPlan	IDataStore, IExerciseCatalog, IProgress (glance)
C7	Attendance & Goals	Gym days, check-ins, weekly goal, week streak	`checkins`	IAttendance	IDataStore, ILogReader, IPlan, IProfile, IDates
C8	Body Tracking	Weigh-ins, goal progress %	`weights`	IBodyTracker, IBodyReader	IDataStore, IProfile, IUnits
C9	InBody Reports	Reports, metrics, image compression	`inbody`	IInBody, IInBodyReader	IDataStore, IUnits
C10	Units & Dates (Shared Kernel)	kg/lb conversion, rounding, date strings, Saturday week math	none	IUnits, IDates	none
C11	UI Kit	Design system: Sheet, Modal, Stepper, UndoToast, PlateBar, focus trap, theme, chart card	none	UI components	none
C12	App Shell & Composition Views	Routing, layout/nav, Dashboard, Exercise detail, Exercise sheet, Charts and Settings pages	none	(top level)	all of the above
C13	Crew & Leaderboard (future)	Opt-in sharing and ranking	`crews` (new)	ICrew	IProgress, IDataStore, IProfile

Dependency direction: C12 -> C3..C9 -> C2 / C10 / C11. No cycles: C5 receives logs as arguments (it does not call C4), C4 uses C5 and C3, C6 uses C3 and C5, C2 receives the uid from the shell (no dependency on C1). See docs/Forge_Component_Spec.docx for the full provided/required lists.

File ownership (current file -> component)

File	Component	Note
`firebase.js`	C2	auth export also used by C1 (split: `db` -> C2, `auth` -> C1)
`AppContext.jsx`	MIXED	auth+profile -> C1; add/put/merge/ensureDoc/upd/del/useCol -
`pages/Login.jsx`, `pages/Settings.jsx` (profile fields)	C1	Settings is a Composition View (C12) that calls IProfile and IAtten
`data/exercises.json`, `data/muscleFolders.js`, `data/popular.js`, `data/videos.json`, `lib/exerciseSearch.js`, `lib/exerciseData.js`, `lib/exerciseSummary.js`	C3	
`lib/exerciseViews.js`	MIXED	libView/customView/docView -> C3; recentExerciseIds, groupLog
`pages/Exercises.jsx`, `exercise/{ExerciseImage,ExerciseMedia,MuscleChips,HowTo,FavoriteButton,MyCustom}.jsx`	C3	MyCustom = custom exercises + user folders (two sub-modules o
`exercise/LogTab.jsx`, `lib/setDrafts.js`	C4	the "Add to today's plan" button inside LogTab moves to C6 (`Adc`
`lib/exerciseStats.js`, `exercise/{ProgressPanel,Glance}.jsx`, `computePRs/epley` from utils	C5	
`pages/Charts.jsx`	C12	Composition View over C5 data
`lib/plan.js`, `data/planTemplates.js`, `lib/usePlan.js`, `lib/usePlanActions.js`, `components/plan/*`, `pages/Plan.jsx`	C6	`usePlan` contains the legacy migration (sub-module)
`weekStart`, `weekStreak`, `WEEK_GOAL`, checkin logic in Dashboard	C7	week math primitives (`weekStart`) live in C10, streak/goal in C
`pages/Body.jsx`, `progressPct`	C8	
`pages/InBody.jsx`, `compressImage`, `IN_METRICS`	C9	

File	Component	Note
utils.js	MIXED	split: KG_PER_LB/r1/ymd/todayStr/fmtDate/shortDate/DAY_NAMES/V -> C10; computePRs/epley -> C5; weekStreak/WEEK_GOAL -> C7 compressImage/IN_METRICS -> C9; COLORS -> C11
components/{Sheet,Modal,Stepper,PlateBar,ChartCard}.jsx, plan/UndoToast.jsx, lib/useFocusTrap.js, styles.css	C11	UndoToast and PlateBar are generic visuals
App.jsx, main.jsx, components/Layout.jsx, pages/Dashboard.jsx, pages/ExerciseDetail.jsx, exercise/{ExerciseSheet,useExerciseSheet}.jsx, plan/TodayCard.jsx	C12	Composition Views: only wire components, no rules

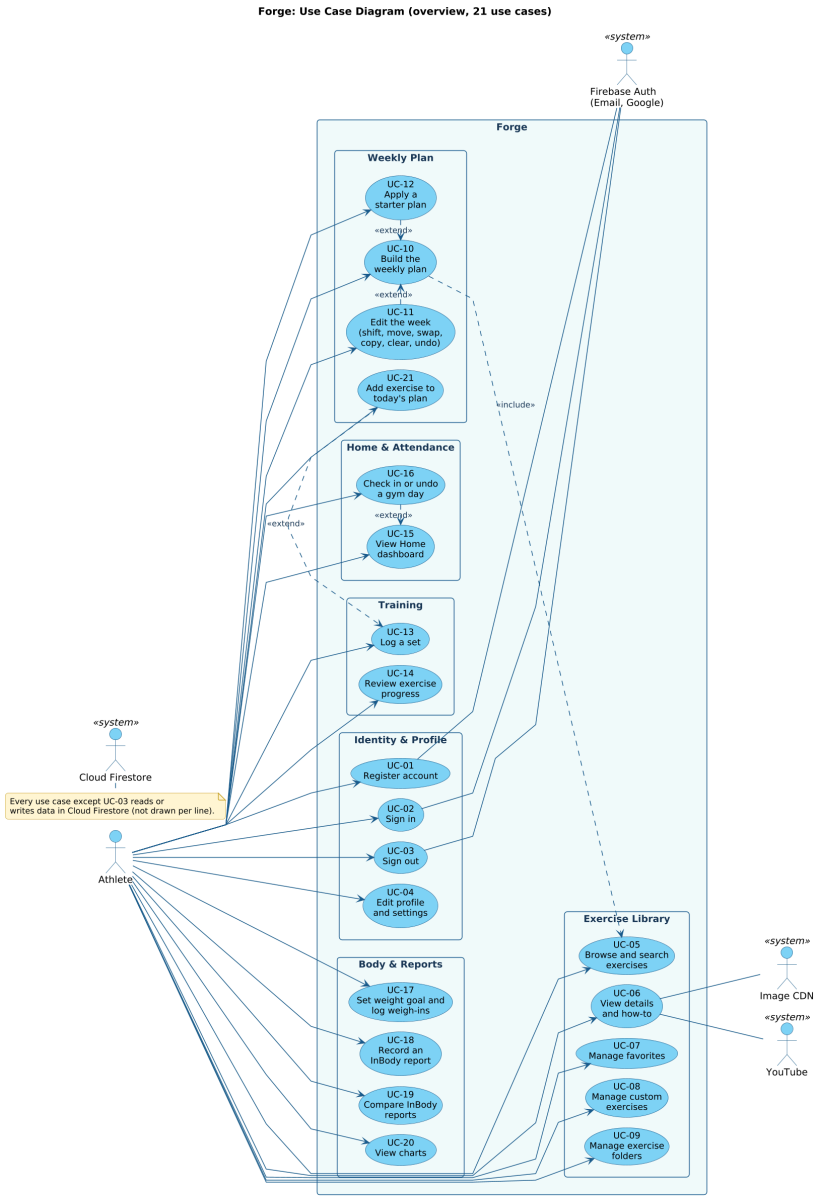
Mixed-responsibility spots found (and how the design fixes them)

1. `AppContext.jsx` mixed auth, profile, units and data access -> C1, C2, C10.
2. `utils.js` mixed six unrelated concerns -> split across C5, C7, C8, C9, C10, C11.
3. `exerciseViews.js` mixed catalog views with log helpers -> C3 / C4.
4. `ExerciseDetail.jsx`, `Dashboard.jsx`, `Settings.jsx`, `Charts.jsx`, `ExerciseSheet.jsx`, `TodayCard.jsx` each combine several components -> declared Composition Views (C12) that only call interfaces.
5. `LogTab.jsx` edited the plan directly -> goes through IPlan (C6).
6. `MyCustom.jsx` holds custom exercises and user folders -> two sub-modules of C3.
7. Weekly goal was split across profile (`weeklyGoal`), plan (`planGoal`) and utils (`WEEK_GOAL`) -> owned by C7, which reads IPlan and IProfile.

Part 4. UML Diagrams

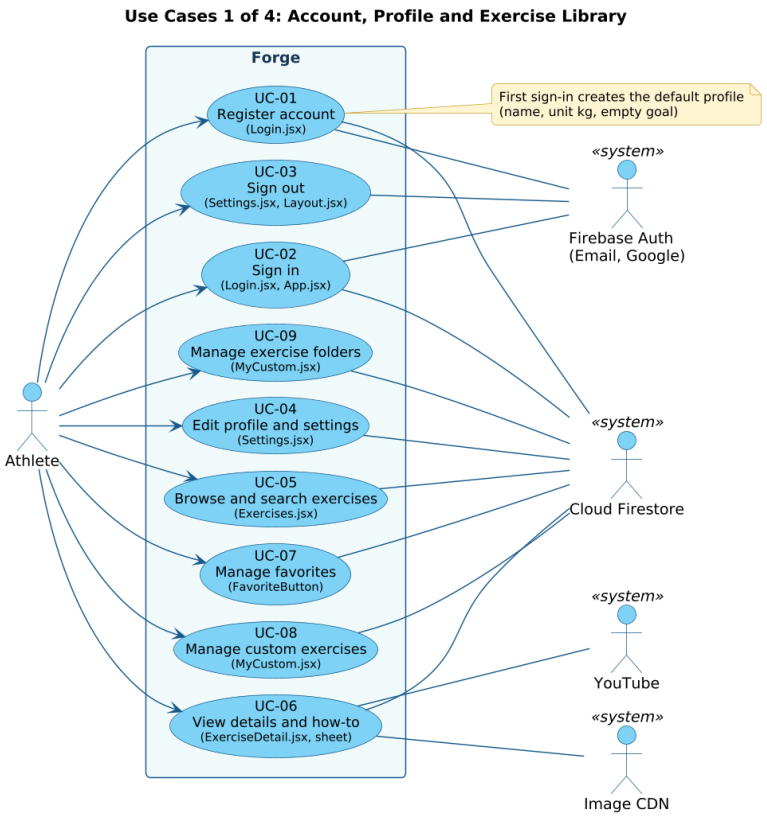
Use case, component, class and sequence diagrams (24 figures).

Figure 1. Use Case diagram 0: overview of all 21 use cases



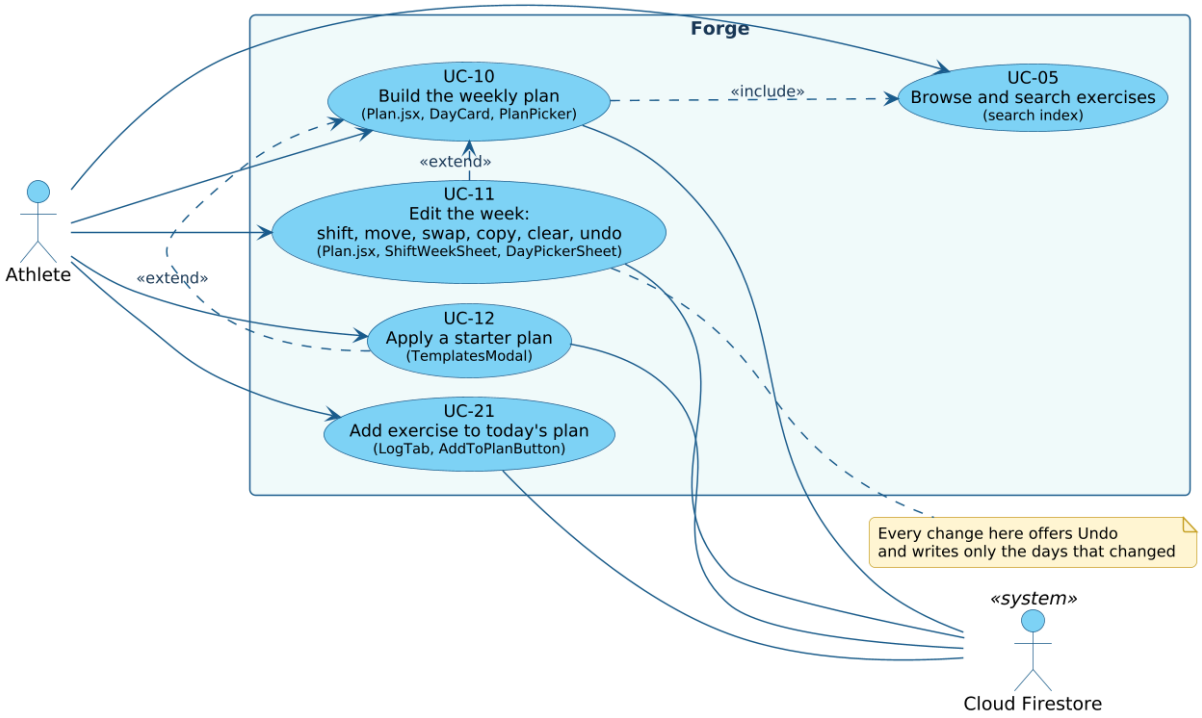
Athlete and the four external systems, grouped by component. Extend and include relations follow the code.

Figure 2. Use Case diagram 1 of 4: account, profile and exercise library



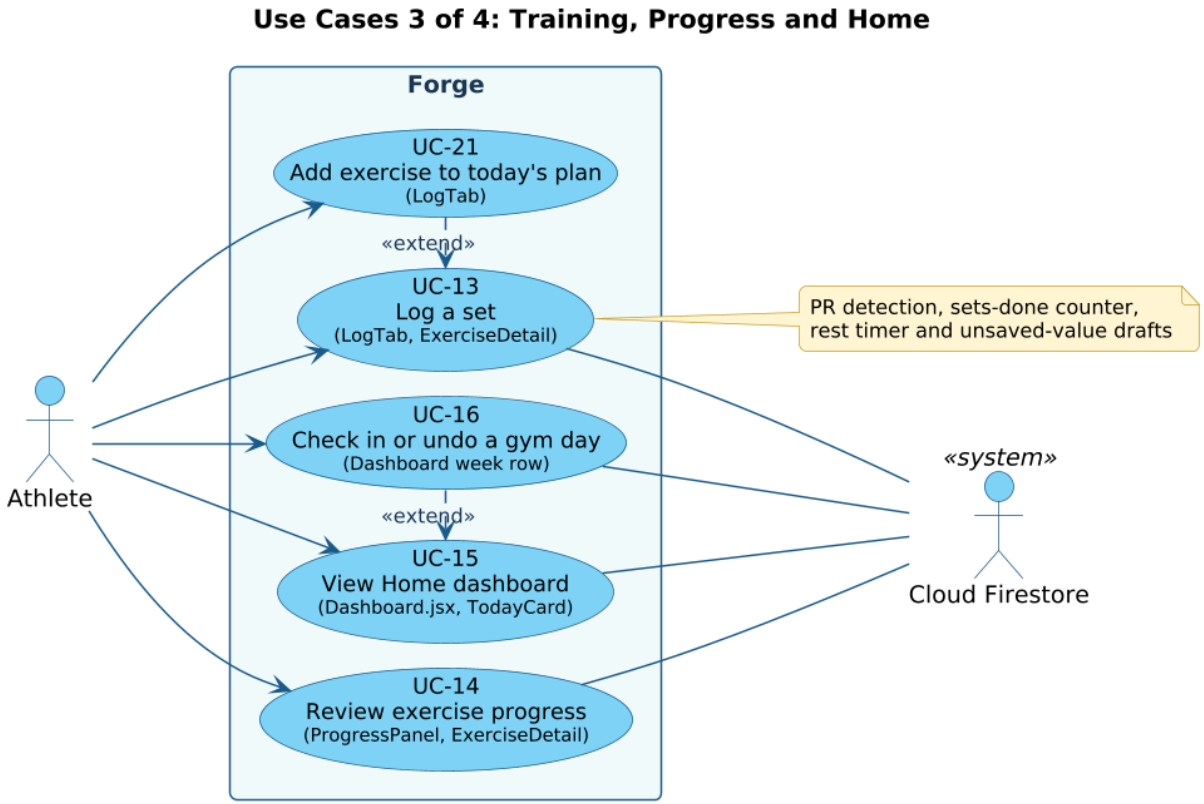
UC-01 to UC-09 with the screens that implement them.

Figure 3. Use Case diagram 2 of 4: weekly plan



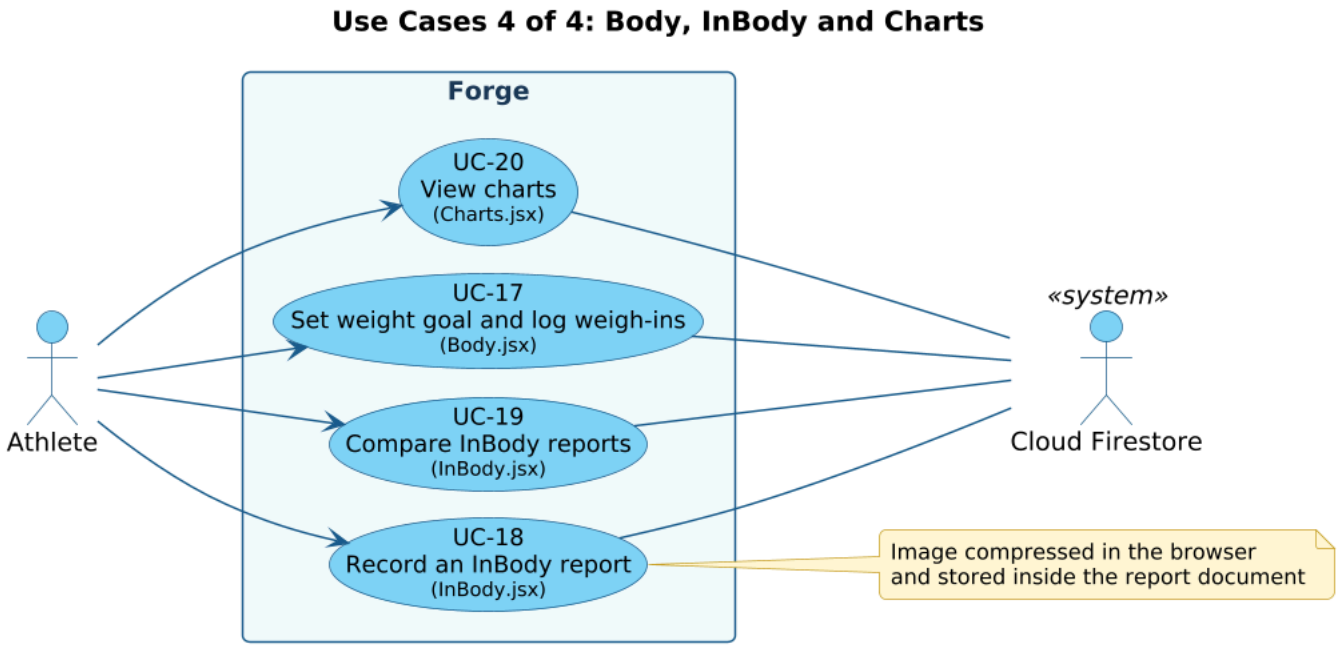
UC-10, UC-11, UC-12, UC-21 and the search it includes.

Figure 4. Use Case diagram 3 of 4: training, progress and Home



UC-13 to UC-16 and UC-21.

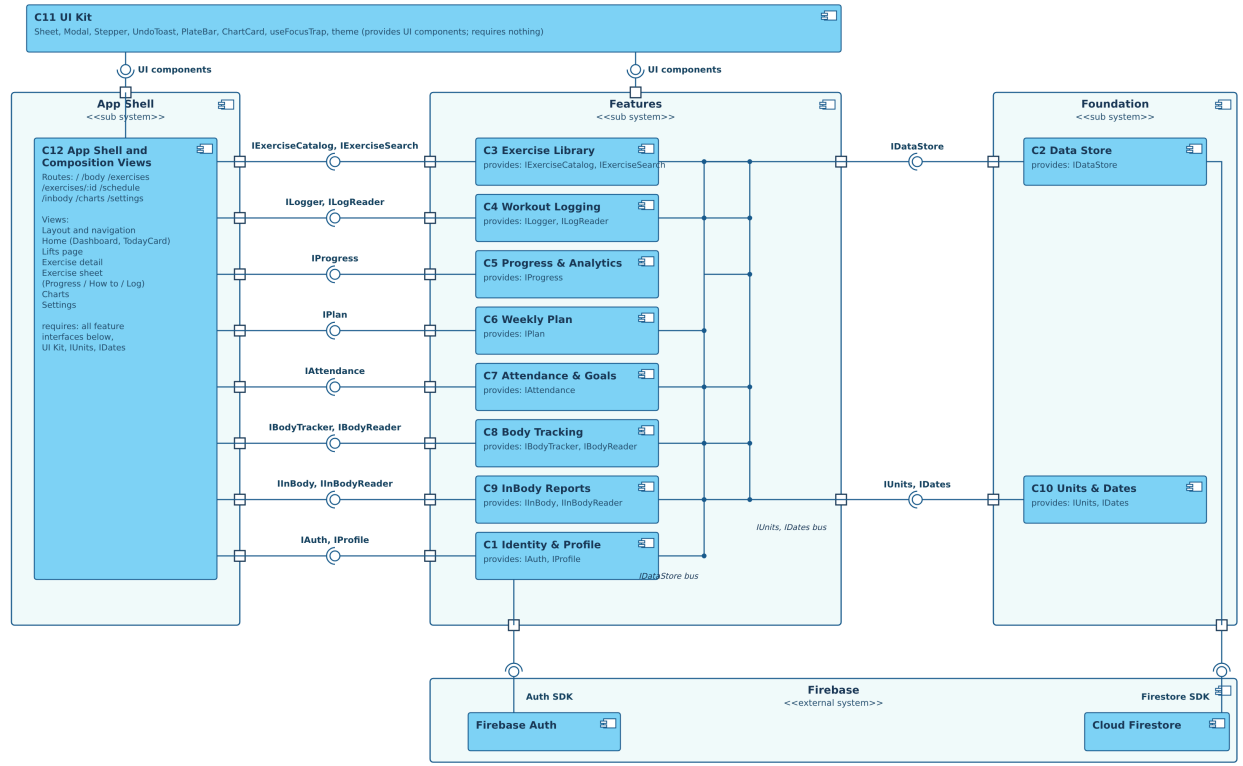
Figure 5. Use Case diagram 4 of 4: body, InBody and charts



UC-17 to UC-20.

Figure 6. Component diagram 0: overview

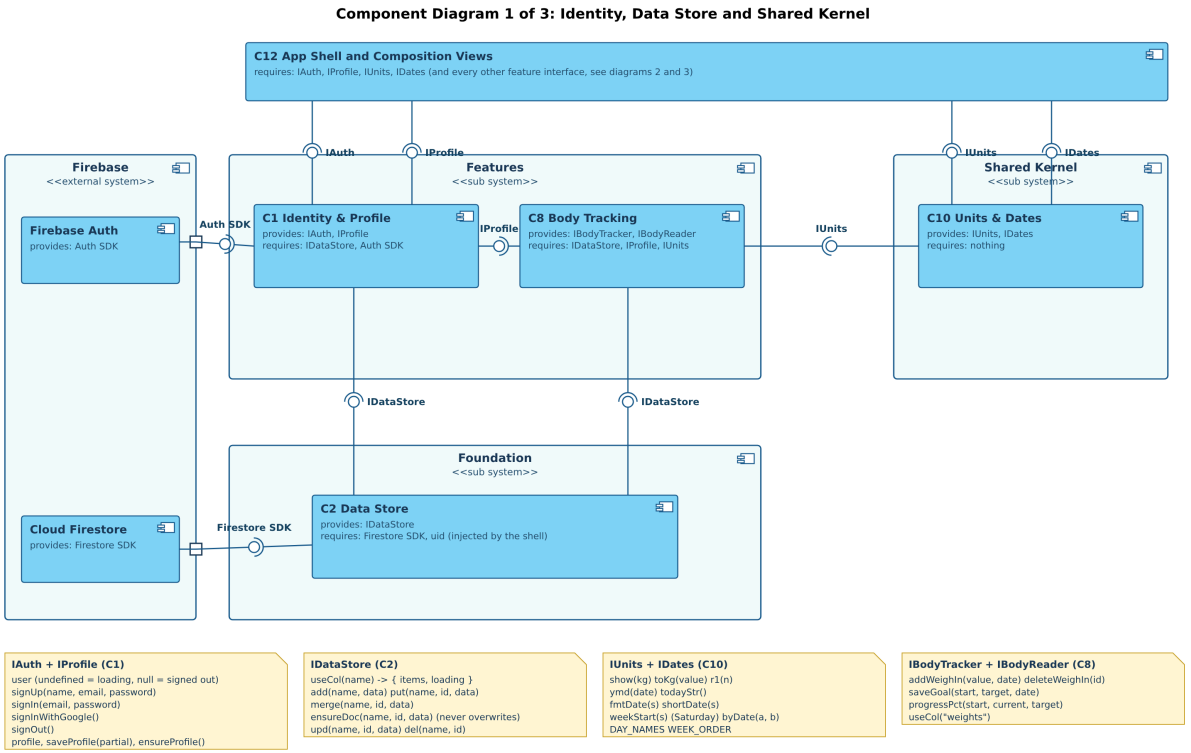
Forge: Component Diagram (overview of all components and their interfaces)



Ball = provided interface, socket = required interface. Squares are ports on a sub system boundary; lines inside a sub system deliver the interface to the components that need it.

Sub systems, ports and provided/required interfaces of all 13 components.

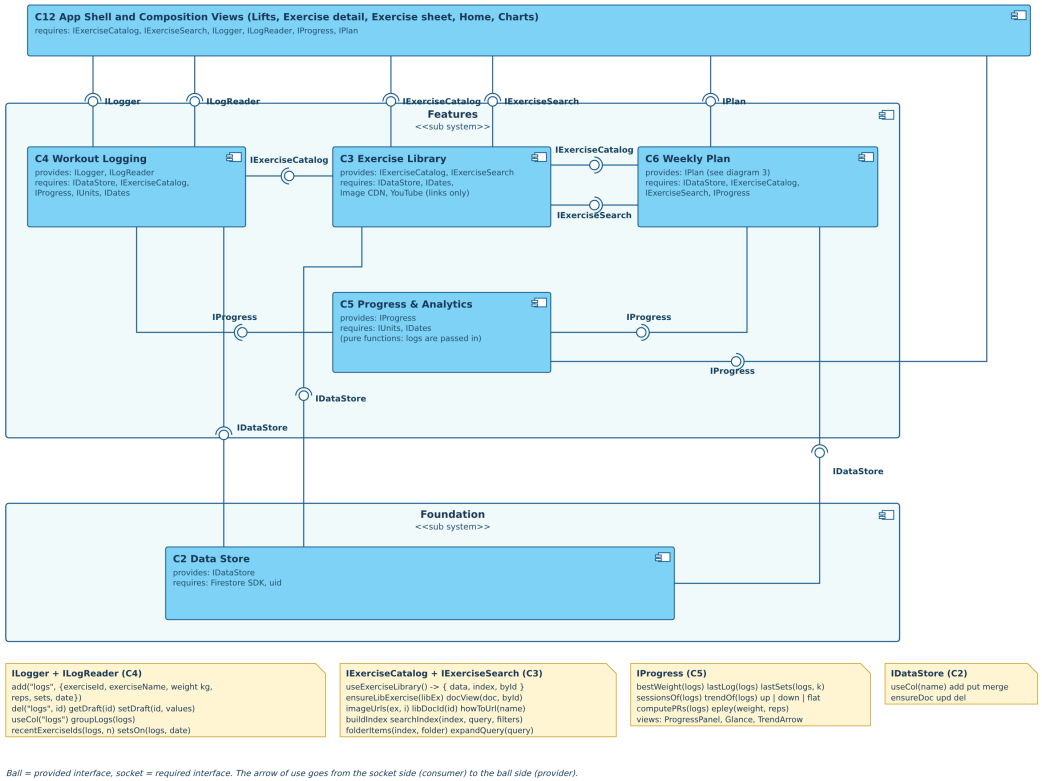
Figure 7. Component diagram 1 of 3: identity, data store and shared kernel



Ball = provided interface, socket = required interface. Lines inside a box show what the component provides and requires.

Ball = provided interface, socket = required interface.

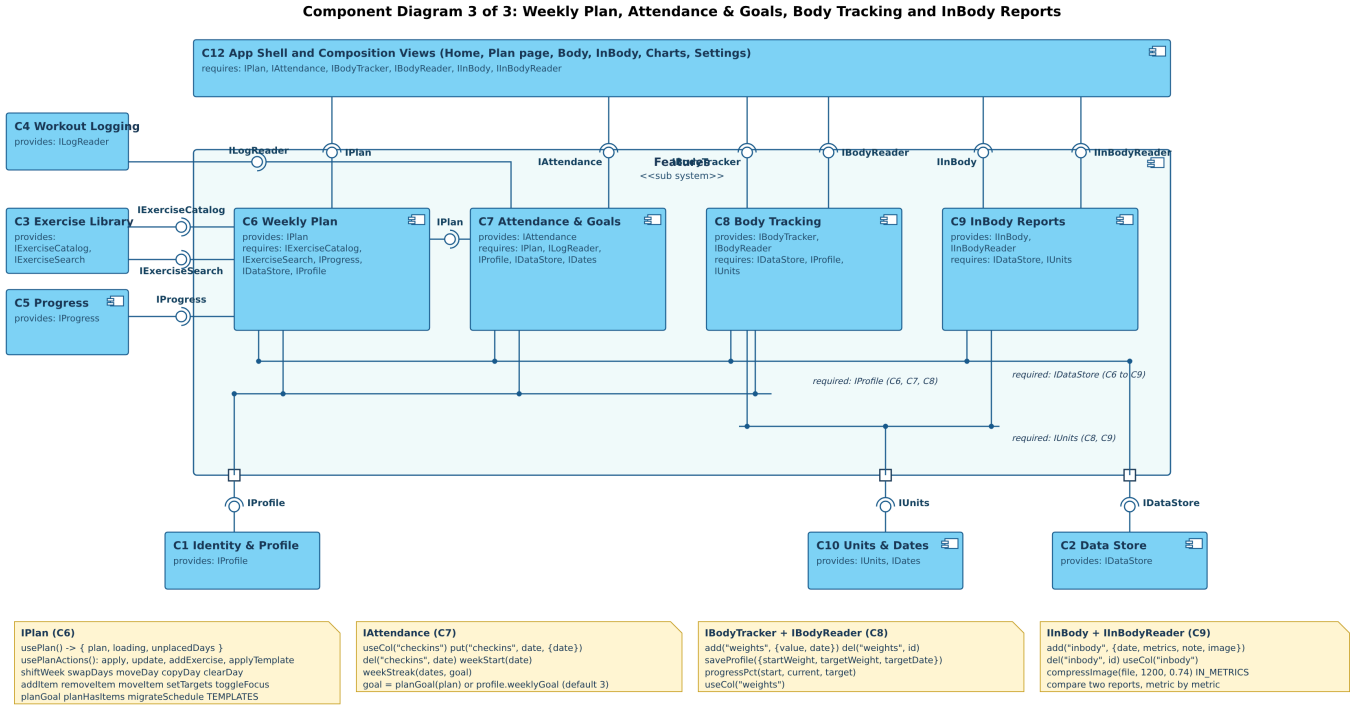
Figure 8. Component diagram 2 of 3: library, logging and progress



Ball = provided interface, socket = required interface. The arrow of use goes from the socket side (consumer) to the ball side (provider).

Interfaces among C3, C4, C5, C6 and the data store.

Figure 9. Component diagram 3 of 3: plan, attendance, body and InBody



Ball = provided interface, socket = required interface. Squares on the Features box are ports; lines from a port inside the box deliver the interface to every component that needs it.

Ports on the Features sub system deliver shared interfaces.

Figure 10. Class diagram 1 of 7: Identity & Profile and Data Store

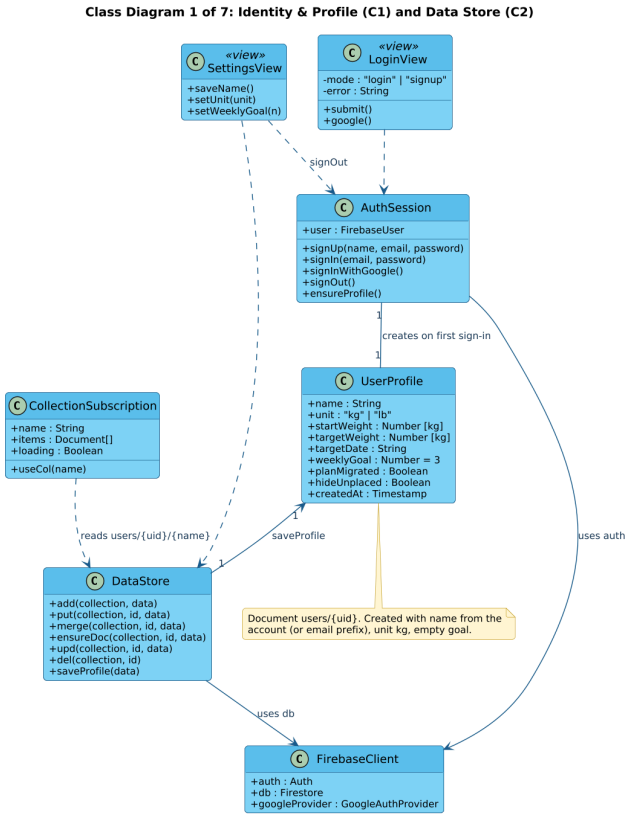


Figure 11. Class diagram 2 of 7: Exercise Library

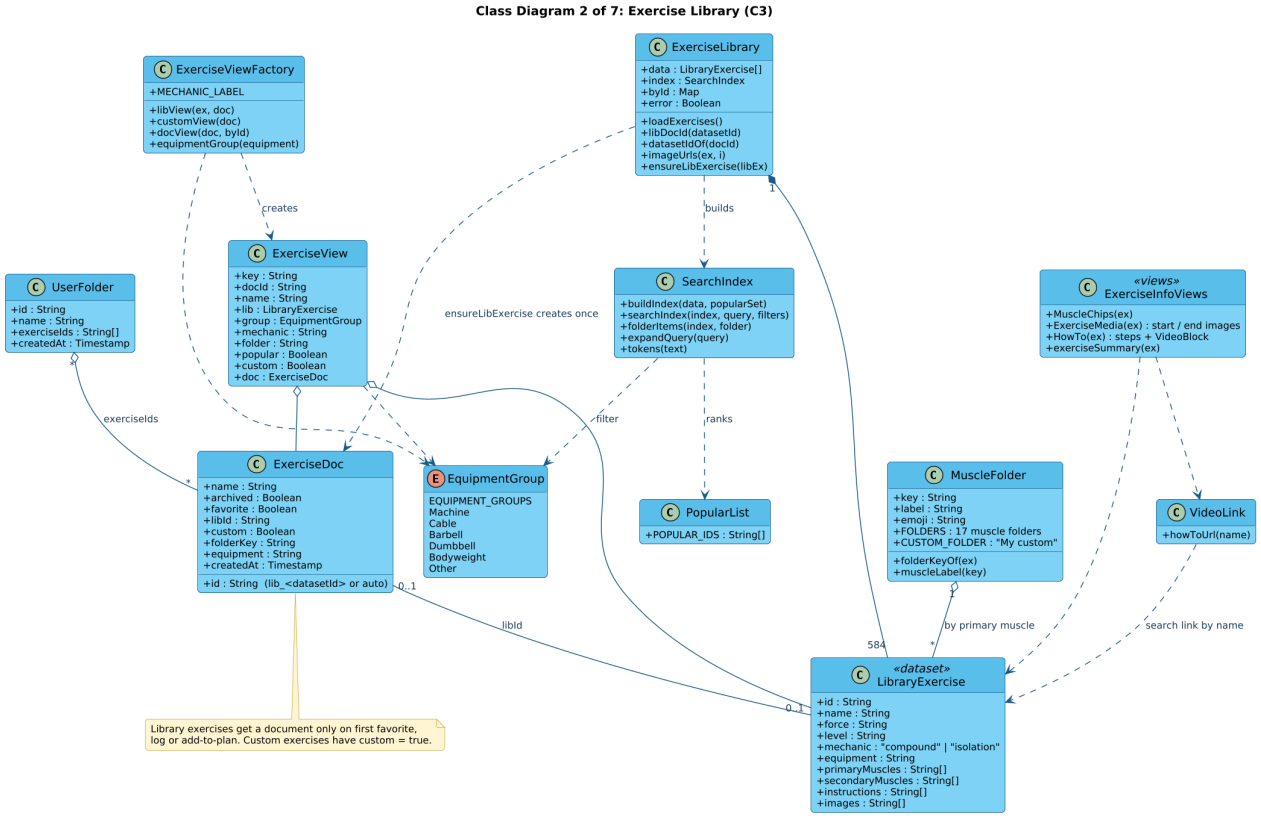


Figure 12. Class diagram 3 of 7: Workout Logging and Progress & Analytics

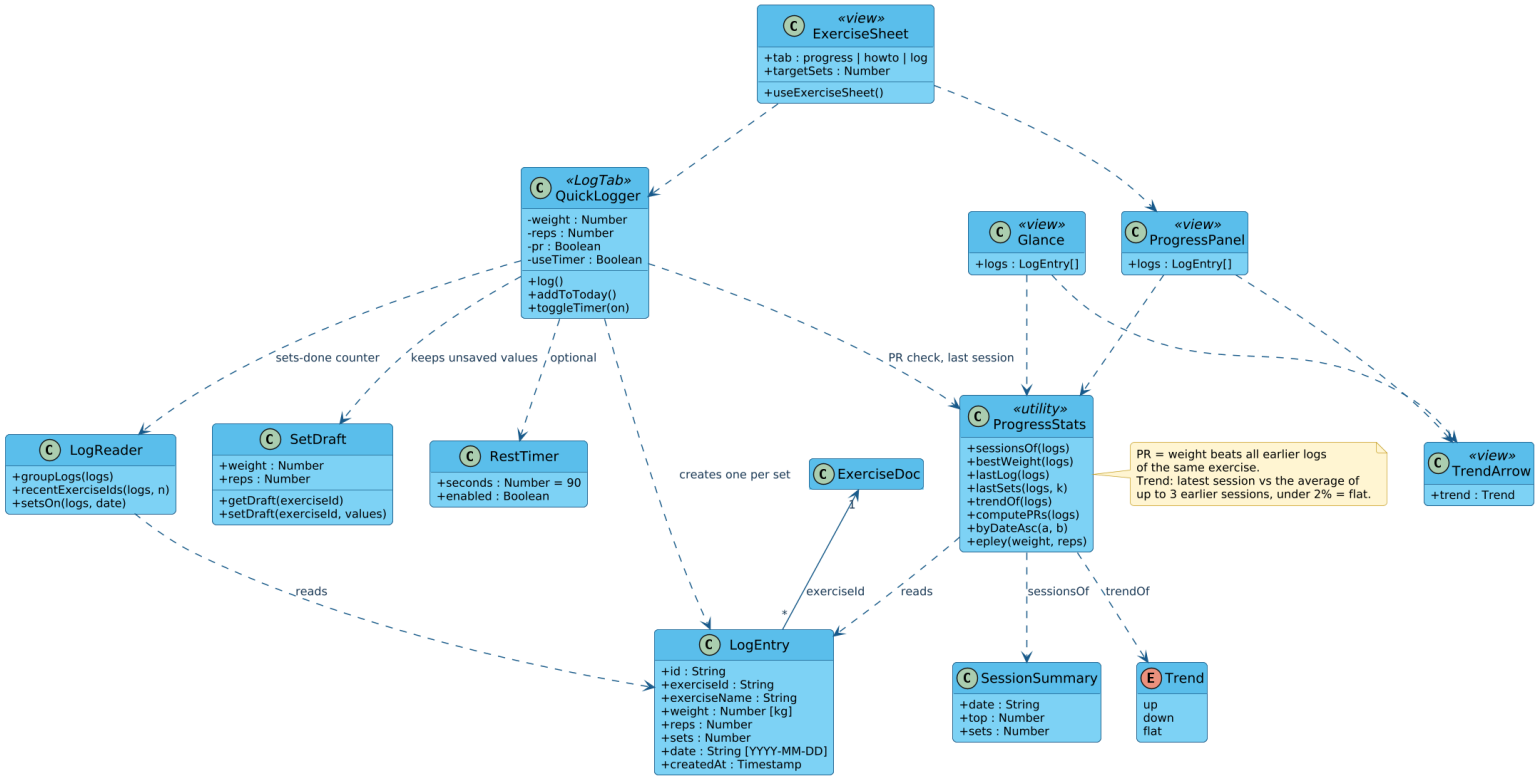


Figure 13. Class diagram 4 of 7: Weekly Plan

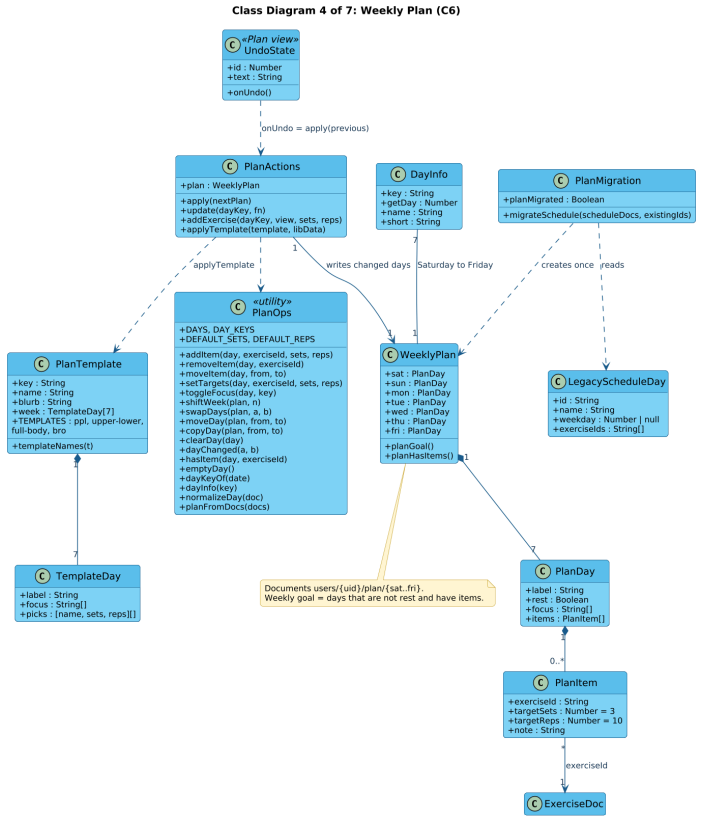


Figure 14. Class diagram 5 of 7: Attendance, Body Tracking and InBody

Class Diagram 5 of 7: Attendance (C7), Body Tracking (C8) and InBody (C9)

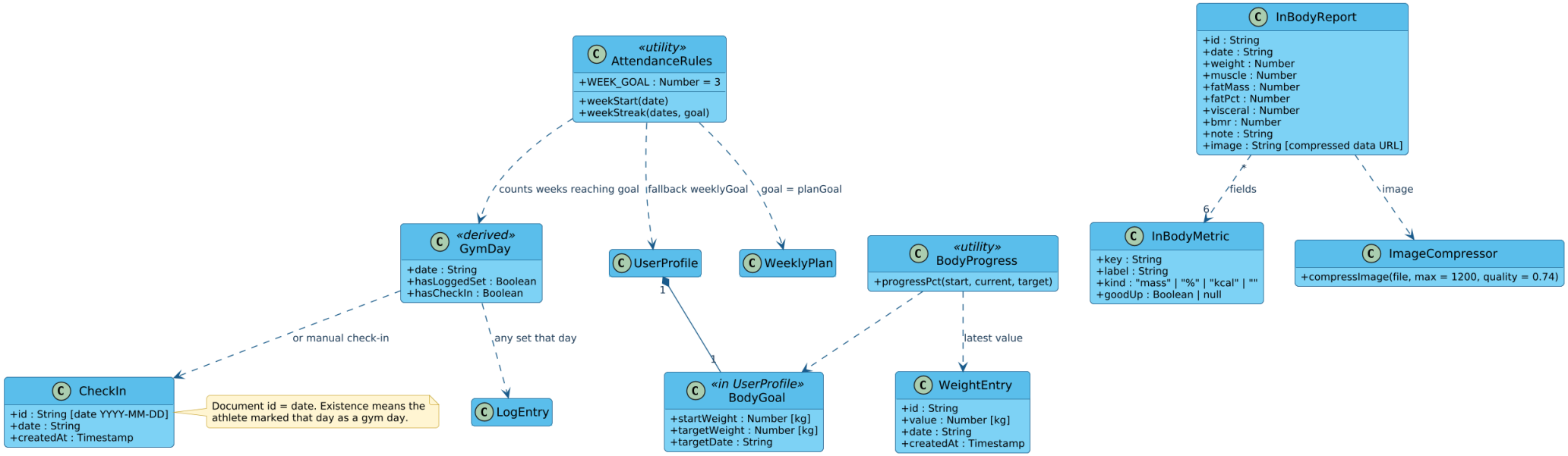


Figure 15. Class diagram 6 of 7: Units & Dates and UI Kit

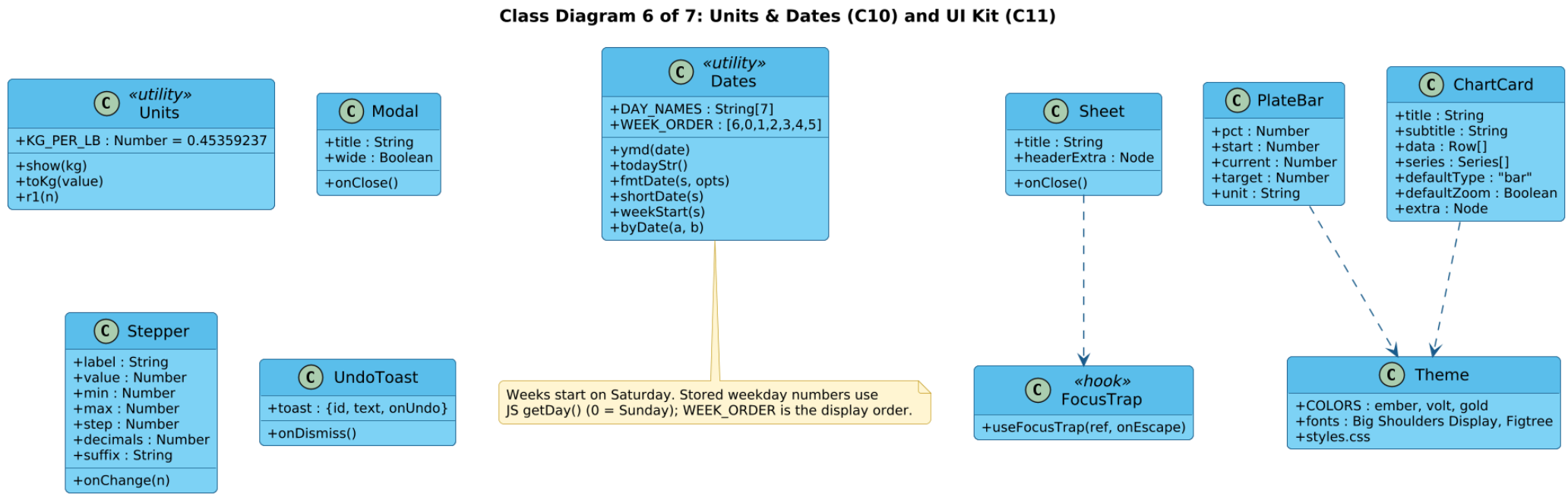
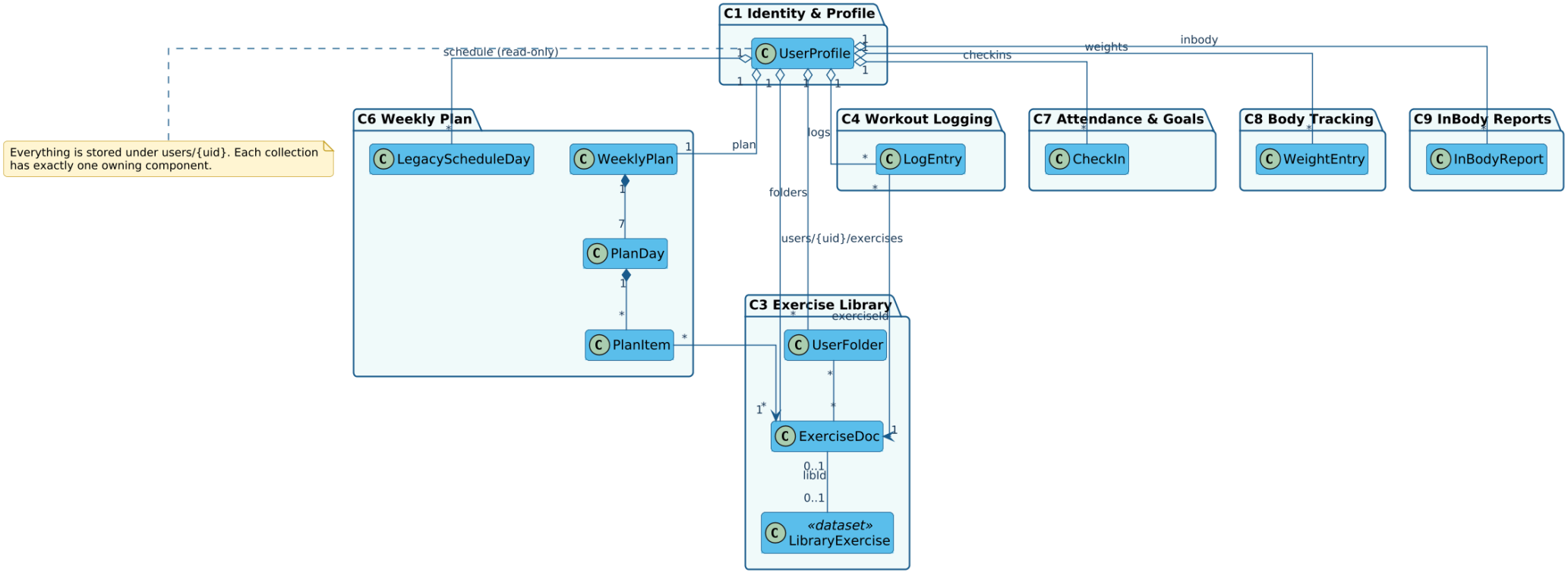


Figure 16. Class diagram 7 of 7: data entities and owners

Class Diagram 7 of 7: Data entities and owners (overview)



Every collection is owned by exactly one component.

Figure 17. Sequence 1 of 8: sign in and first-time profile (UC-01, UC-02)

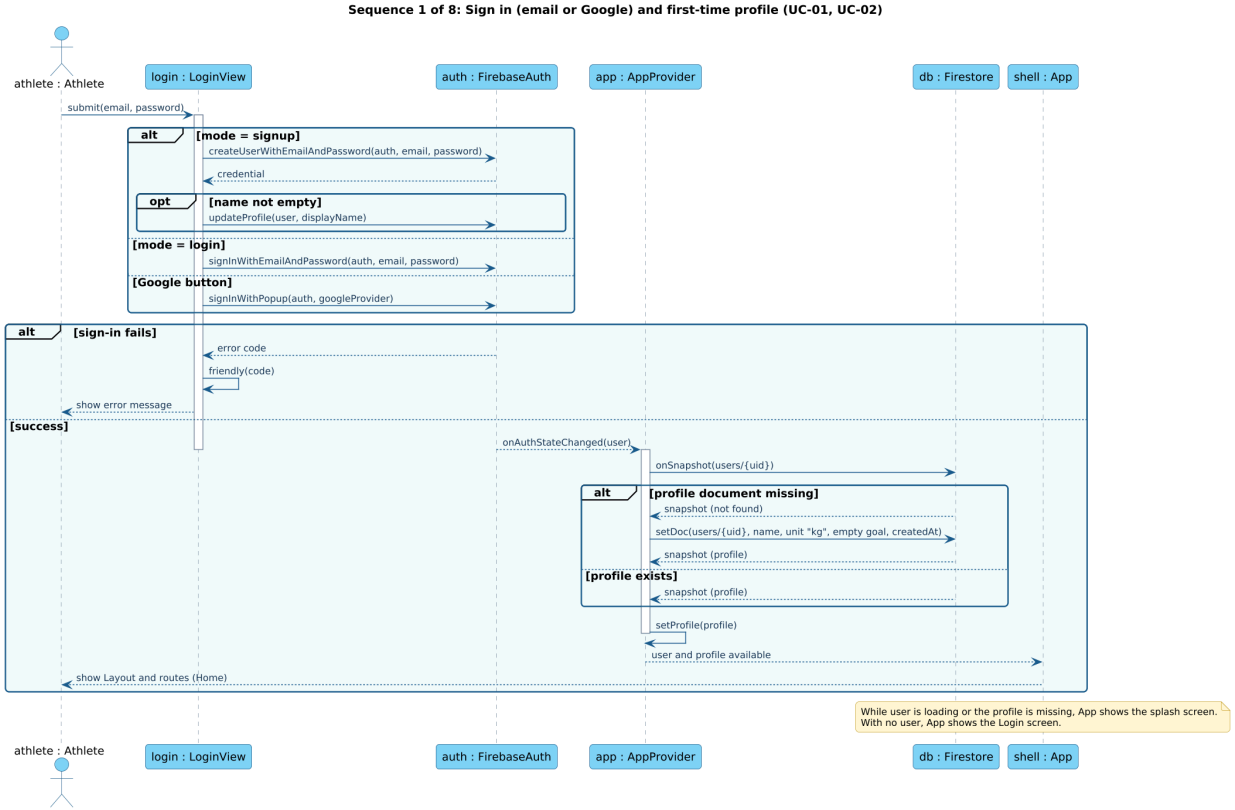


Figure 18. Sequence 2 of 8: log a set (UC-13)

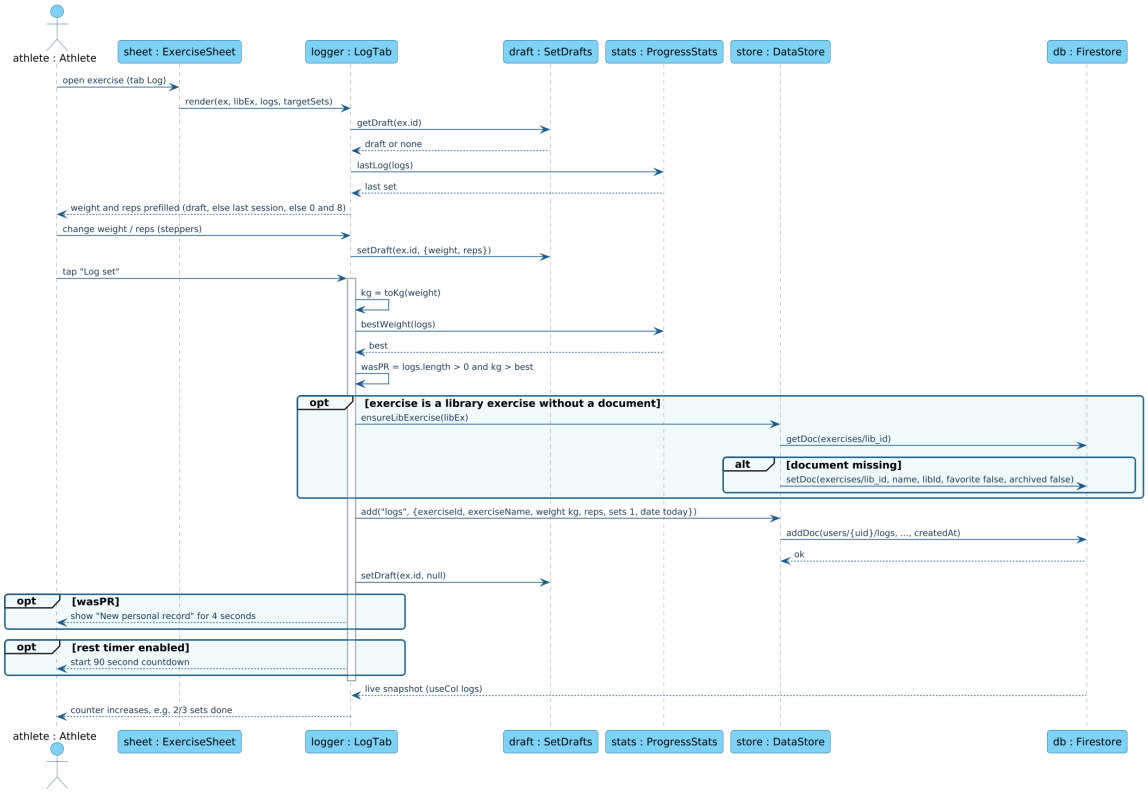


Figure 19. Sequence 3 of 8: add an exercise to a plan day (UC-10)

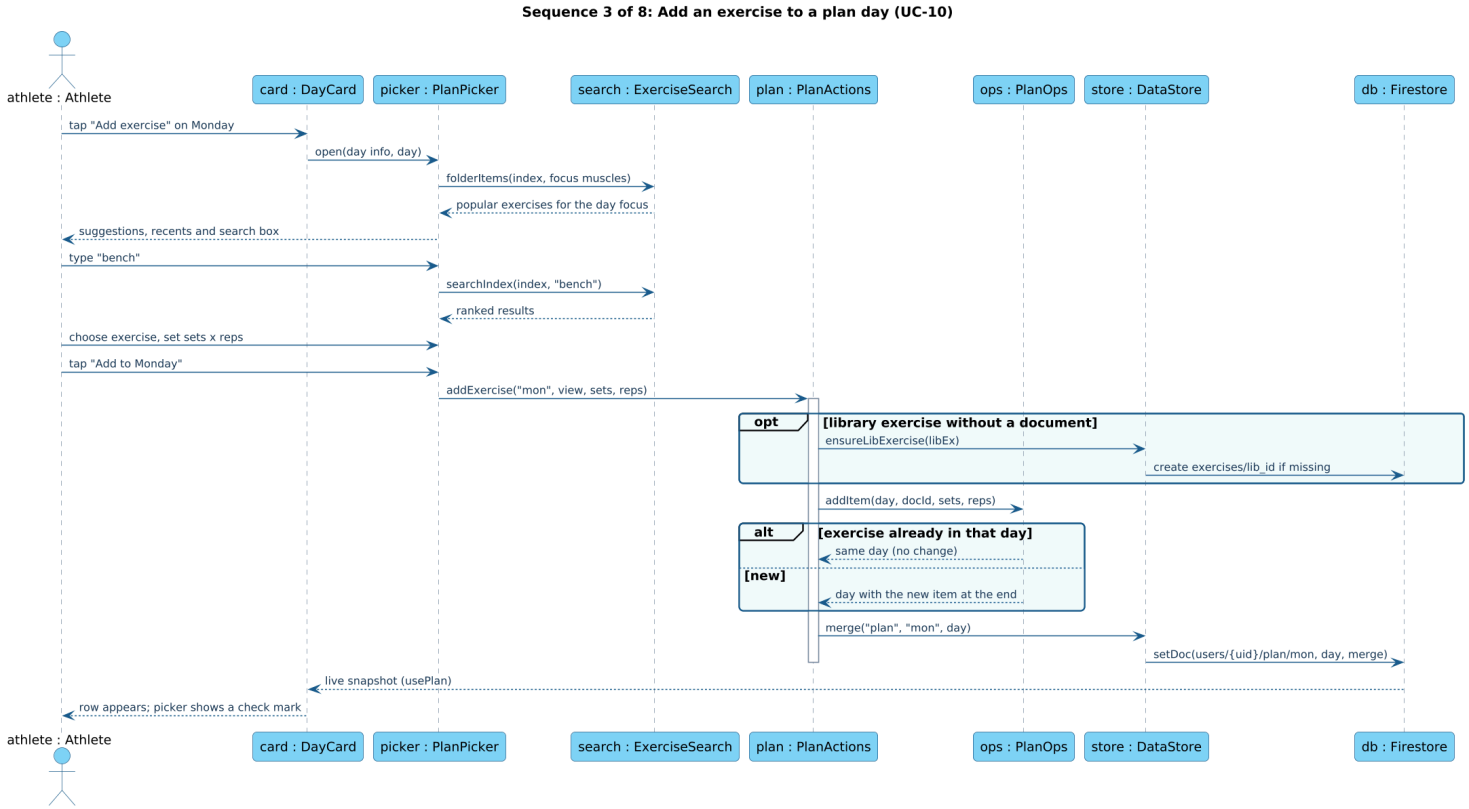


Figure 20. Sequence 4 of 8: shift the whole week and undo (UC-11)

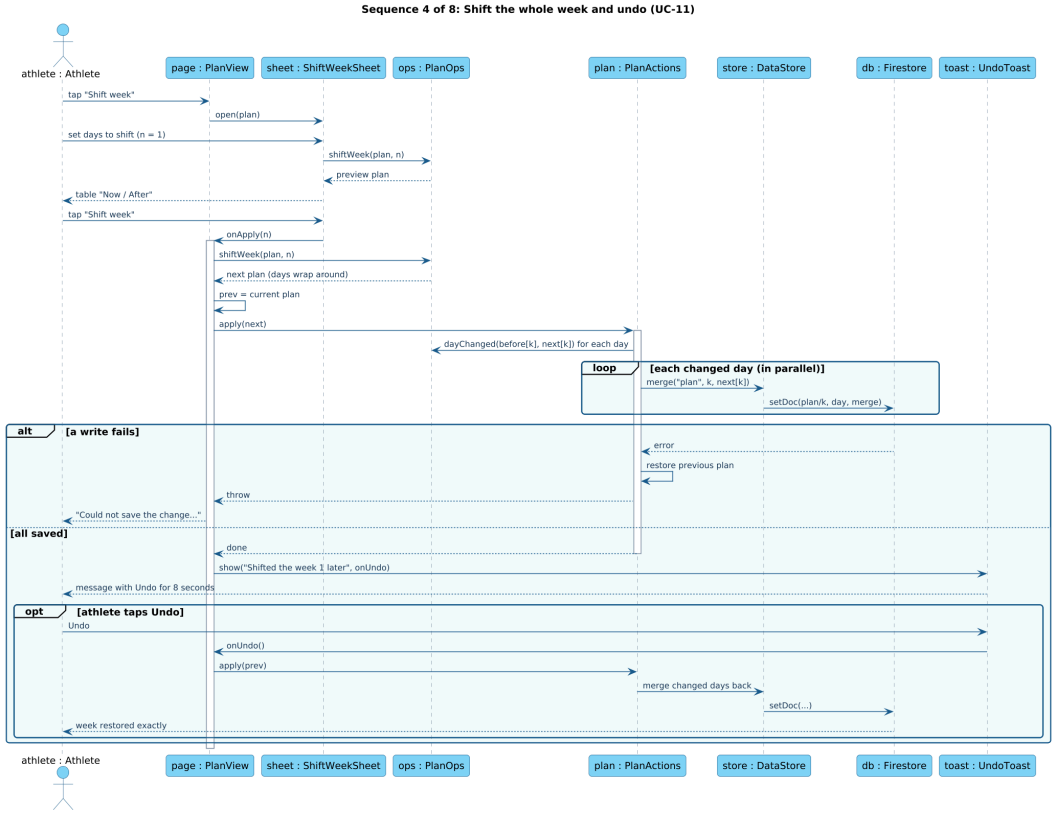


Figure 21. Sequence 5 of 8: one-time schedule migration (UC-10)

Sequence 5 of 8: One-time migration of the old schedule into the plan (UC-10)

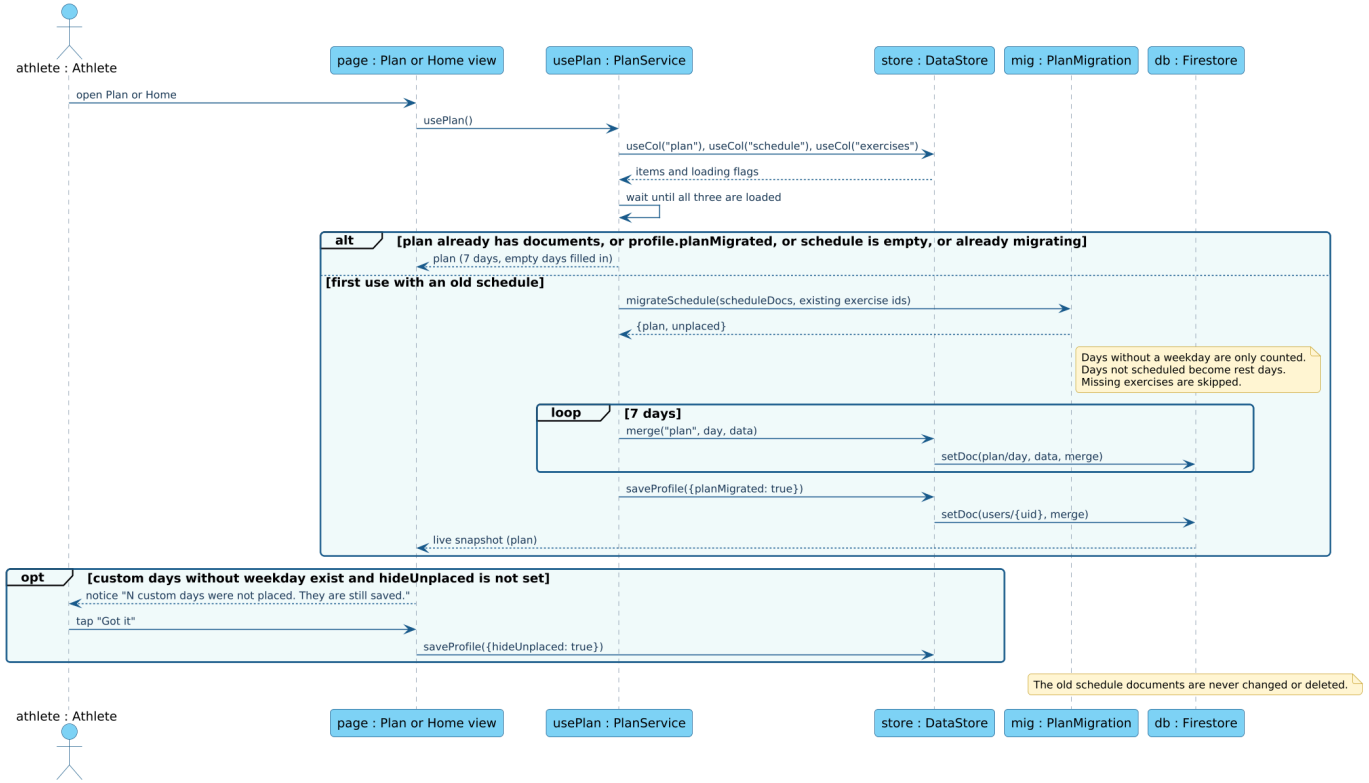


Figure 22. Sequence 6 of 8: Home, weekly goal and check-in (UC-15, UC-16)

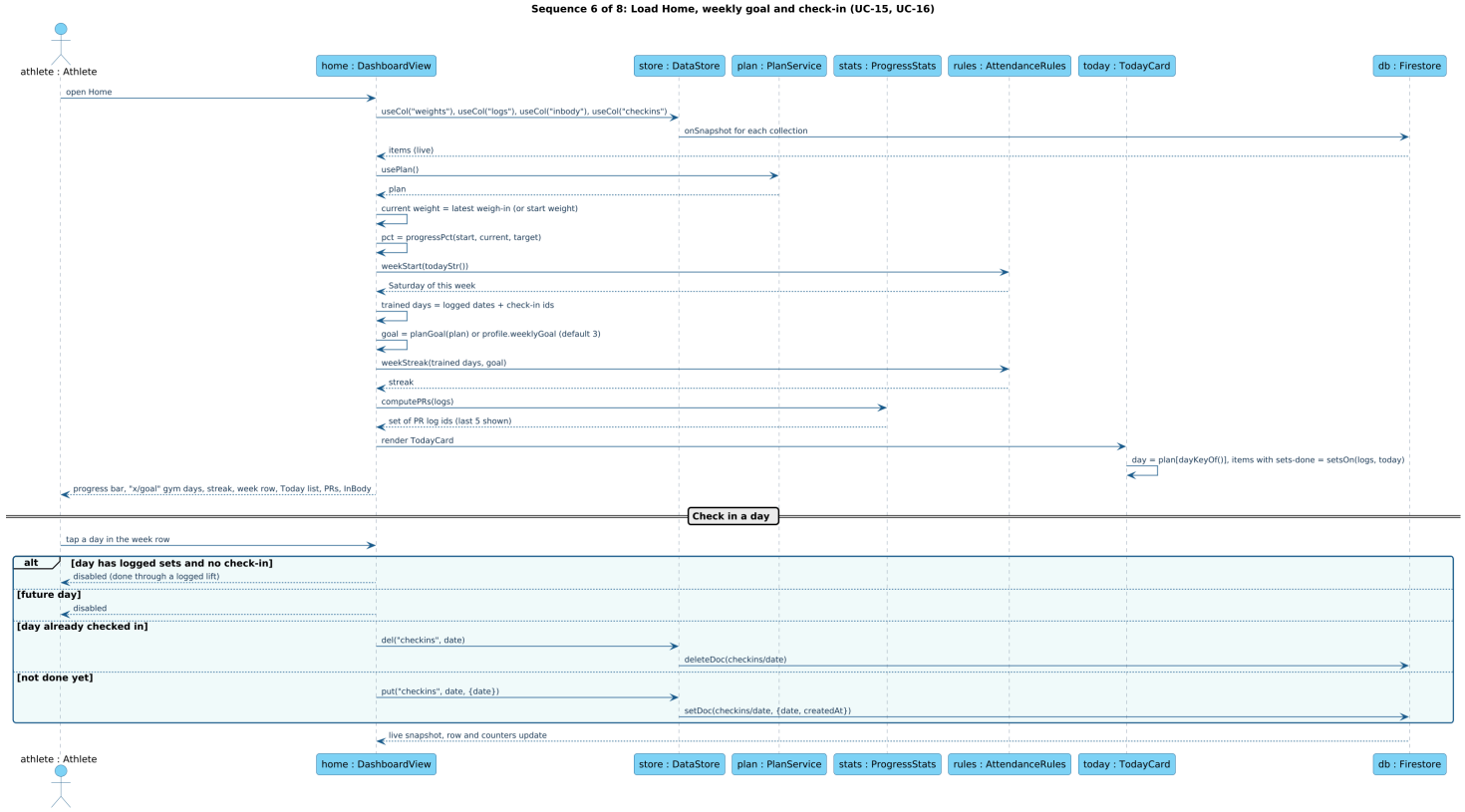


Figure 23. Sequence 7 of 8: library search, details and favorites (UC-05 to UC-07)

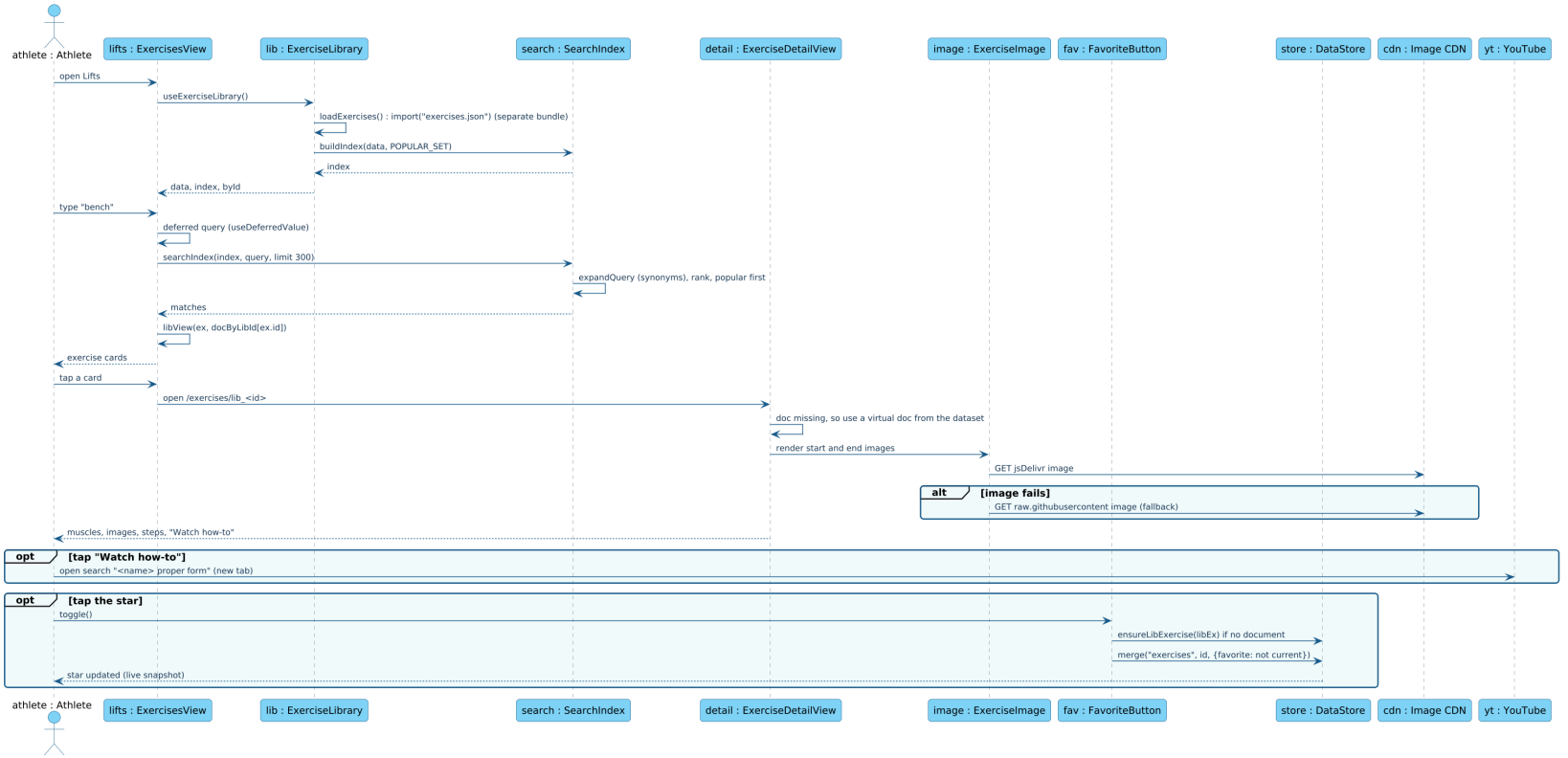


Figure 24. Sequence 8 of 8: record an InBody report (UC-18)

